Year: 2026-2027 **Sem V-NEP**

Course: Agile Software Development & DevOps Lab

Class: T1 **Date of Submission: 12 / 10 / 2026**

A Mini Project Report on

Global Logistics Tracking System

A Shipment Management and Tracking Application Delivered Through an Agile and DevOps Pipeline

Submitted in partial fulfilment of the requirements of the course

Agile Software Development & DevOps Lab

Semester V (NEP) · Academic Year 2026–2027

Submitted by

Name	Roll No.
Vineet Gajwani	2401028
Aryan Kashid	2401042
Piyush Gupta	2401033

Under the guidance of

Dr. Sanober Shaikh

Department of AI&DS

Thadomal Shahani Engineering College

2026–2027

CERTIFICATE

This is to certify that the mini project entitled **"Global Logistics Tracking System: A Shipment Management and Tracking Application Delivered Through an Agile and DevOps Pipeline"** is a bona fide work of **Vineet Gajwani (2401028)**, **Aryan Kashid (2401042)** and **Piyush Gupta (2401033)**, submitted in partial fulfilment of the requirements of the course Agile Software Development & DevOps Lab, Semester V, during the academic year 2026–2027.

_____ _____

Project Guide **Head of Department**

Date: _____ Place: _____

ABSTRACT

Shipment operations depend on knowing, at any moment, which customer a consignment belongs to, how it is moving and what its current status is. Where this information lives in spreadsheets and messages, shipments are mislaid, updates arrive late and customers have no single place to look. The Global Logistics Tracking System (branded LogistiQ in its interface) addresses this with a web application in which an operator registers customers, creates shipments against them, records tracking events and looks up the full timeline of any shipment by its identifier, while a table of all shipments gives an overview of the operation.

The application, built with HTML, CSS and JavaScript on the front end, Node.js and Express on the server and a MySQL relational database, was used as the vehicle for applying the seven experiments of the Agile Software Development & DevOps Lab to a single product. The work was planned in Jira as an epic with six user stories in one sprint, and each story corresponds to one lab experiment: Git version control, Docker, Docker Compose, a Jenkins continuous-integration pipeline, automated deployment through Jenkins, Jira-based project management and application monitoring. The system is stored on GitHub, packaged as a Docker image, run together with MySQL as a Docker Compose stack and built and deployed by a declarative Jenkins pipeline of eight stages. A Nagios Core container added to the same stack checks the availability of the web application and the database port.

Eight pipeline executions were recorded. The first four failed while the environment and the pipeline were being configured; the last four, builds 5 to 8, succeeded, and build 8 delivered a visible change to the dashboard heading from a commit to the running container through the pipeline. The Compose start-up also showed an application container that initially could not reach a database that was still initialising, which is a typical example of the inspect-and-adapt cycle that the Agile practices are intended to encourage.

Keywords: Agile, Scrum, Jira, Git, GitHub, Docker, Docker Compose, Jenkins, CI/CD, Nagios, Node.js, MySQL

CONTENTS

Chapter	Title
1	Introduction
2	Requirements and Agile Planning (Experiment 6)
3	System Design
4	Application Implementation
5	DevOps Implementation (Experiments 1–5 and 7)
6	Testing and Verification
7	Results and Discussion
8	Conclusion and Future Scope
	References
A	Appendix: Running the Project

1. Introduction

1.1 Background

Logistics companies move goods across cities, countries and continents, and every consignment passes through several hands and several locations before it reaches its destination. The people who handle it need to record who the customer is, what is being shipped and when, and what has happened to it along the way; the customer needs to ask where the shipment is and receive a reliable answer. When these records are scattered, tracking a single shipment means asking several people, and nobody has a view of the whole operation.

The Agile Software Development & DevOps Lab introduced, through seven experiments, the practices used to deliver software reliably: version control, containers, multi-container orchestration, continuous integration, automated deployment, Agile project tracking and monitoring. Each experiment is normally performed on a small, separate exercise. In this mini project all seven were applied to one existing application, so that the way the practices depend on each other becomes visible: the repository feeds the pipeline, the pipeline builds the container images, the containers form the deployed system and the monitoring tool observes that system, while the project board records the plan and progress of all of it.

1.2 Problem Statement

To provide a web application that lets an operator manage customers and shipments and track each shipment through its events, and to deliver it through an Agile, automated pipeline in which the work is planned as a backlog in Jira, versioned in Git and GitHub, packaged with Docker, built and deployed by Jenkins, run with Docker Compose and observed by a monitoring tool.

1.3 Objectives

1. Allow an operator to add customers and create shipments associated with them, including shipping date and weight.
2. Allow any shipment to be tracked by its identifier, showing a timeline of tracking events and its current status.
3. Provide an overview table of all shipments with quick access to tracking.
4. Plan the work in Jira as an epic with prioritised stories and a sprint, and move each story through To Do, In Progress and Done.
5. Keep the source code under Git and publish it to a GitHub repository.
6. Containerise the application with Docker and run it together with MySQL using one Docker Compose command.
7. Build, validate and deploy the application automatically through a Jenkins declarative pipeline stored in the repository.
8. Monitor the availability of the running application and database with Nagios Core.

1.4 Scope

The system serves a single logistics operator and is accessed through a web browser. Its functions are customer management, shipment creation, shipment tracking and listing; user authentication and payment are outside the scope of this project. The deployment target is a single machine running Docker. The same repository, Compose file and Jenkinsfile are intended to be portable to any host that has Docker installed.

1.5 Mapping of Lab Experiments to the Project

Exp	Lab topic	Application in this project	Section
1	Version control with Git	Local repository, initial commit, publication to GitHub, later commits for each increment	5.1
2	Containerisation with Docker	Dockerfile and .dockerignore for the Node.js application; image build and container run	5.2
3	Docker Compose	Two-service stack (application and MySQL) with schema available in the database container	5.3
4	Jenkins CI pipeline	Declarative Jenkinsfile with checkout, tool verification, install, validation and image build stages	5.4
5	Automated deployment	Deploy and Verify stages added; a dashboard change pushed to GitHub and deployed by Jenkins	5.5
6	Agile project lifecycle	Jira project with epic, six stories, sprint and board	2
7	Monitoring	Nagios Core container added to the Compose stack; HTTP and TCP checks	5.6

Table 1.1: Lab experiments and where each is applied

2. Requirements and Agile Planning

2.1 User Roles

Role	Goal	Main screens
Logistics operator	Register customers, create shipments and keep shipment records current	Add Customer, Create Shipment, All Shipments
Customer or support agent	Find out where a shipment is and what has happened to it	Track Shipment (search by shipment ID)
Manager	Get an overview of current shipments	Dashboard, All Shipments

Table 2.1: User roles

2.2 Functional Requirements

- The operator can add a customer to the system.
- The operator can create a shipment that belongs to a customer, with a shipping date and a weight.
- A shipment can be tracked by its identifier, and the result shows its tracking timeline and status changes.
- All shipments are listed in a dynamic table, with a quick action that opens the tracking view.
- Data are stored in a relational MySQL database that has primary and foreign key constraints and sample data for immediate testing.
- The interface is a single-page application with a Dashboard, Add Customer, Create Shipment, Track Shipment and All Shipments views.

2.3 Non-Functional Requirements

- **Portability:** the complete system, application and database, must start with a single `docker compose up --build` on any machine that has Docker.
- **Reliability:** the application image must be built reproducibly, and every change must be verified automatically before the application is redeployed.
- **Security:** database credentials are supplied through environment variables read from a `.env` file; a `.dockerignore` file keeps it out of the Docker build context.
- **Maintainability:** build and deployment steps are stored in the repository as code (Dockerfile, docker-compose.yml, Jenkinsfile).
- **Observability:** the availability of the web application and of the database port must be checked continuously by a monitoring tool.

2.4 How Agile Practices Were Applied

The main goal of the project was to deliver the system through Agile practice, and Experiment 6 (Agile project lifecycle) was therefore treated as the frame for all the other experiments rather than as a separate exercise. The following practices were followed.

- **Backlog and epic.** The whole piece of work was captured as one Jira epic, SCRUM-5 *Logistics Management System*, whose description covers version control, containerisation, continuous integration, deployment, project management and monitoring. Each lab experiment became a user story under that epic (Table 2.2).

- **Prioritisation.** The five stories that form the delivery chain (Git, Docker, Jenkins CI, Jenkins deployment and Jira) were given High priority, because each one is a prerequisite of the next. Monitoring, which needs a running deployed system, was given Medium priority and scheduled last.
- **Sprint with a goal.** The six stories were placed in a single sprint named *MPR DevOps Implementation* whose goal was to take the existing application from a local project to a monitored, automatically deployed system.
- **Incremental delivery of working software.** Every increment left the application in a working state. The Git history records the checkpoints (for example, *Global Logistics and Tracking*, *Done till exp 3*, *Add automated application deployment*, *Update Logistics application frontend*), and the application was run after each of them.
- **Visible workflow.** Stories moved across the Jira board columns To Do, In Progress and Done as the experiments progressed, so that the state of the project was visible at any moment.
- **Frequent integration and fast feedback.** From Experiment 4 onwards every push could be checked by the pipeline, which reported success or failure in a few minutes.
- **Inspect and adapt.** The failed pipeline executions and the database connection error seen at container start-up were treated as feedback: the cause was investigated, the configuration was corrected and the change was verified in the next run (Section 7.2).
- **Evidence at review.** Screenshots of every step were attached to the Jira story for the project-management experiment, so the sprint could be reviewed against concrete evidence.

2.5 Product Backlog

Table 2.2 lists the epic and the six stories that make up the product backlog of the Jira project *Global Logistics & Tracking System*. Every story carries the label *mpr* and the label *devops*, and is linked to the epic as its parent.

Key	Type	Summary	Priority	Labels	Exp
SCRUM-5	Epic	Logistics Management System	—	—	All
SCRUM-6	Story	Implement Git Version Control	High	git, github	1
SCRUM-7	Story	Containerize Logistics Application Using Docker	High	docker	2, 3
SCRUM-8	Story	Configure Jenkins CI Pipeline	High	ci, jenkins	4
SCRUM-9	Story	Deploy Logistics Application Using Jenkins	High	deployment, jenkins	5
SCRUM-10	Story	Manage Project Using Jira	High	agile, jira	6
SCRUM-11	Story	Implement Application Monitoring	Medium	monitoring	7

Table 2.2: Product backlog of the Jira project

Key	Description recorded in Jira
SCRUM-6	Set up Git version control for the Global Logistics & Tracking System and maintain the project source code in GitHub.
SCRUM-7	Create a Docker image for the system and configure the application to run inside a Docker container.
SCRUM-8	Configure Jenkins for automated building and deployment of the system.
SCRUM-9	Use Jenkins to automate the deployment of the system.

Key	Description recorded in Jira
SCRUM-10	Use Jira for Agile project management, issue tracking, sprint planning and monitoring the progress of the system.
SCRUM-11	Implement application monitoring for the deployed system using the monitoring tools specified for Experiment 7.

Table 2.3: Story descriptions as recorded in Jira

Figure 2.1 shows the epic, and Figures 2.2 to 2.7 show each story as recorded in Jira, with its priority, description, labels and parent epic.

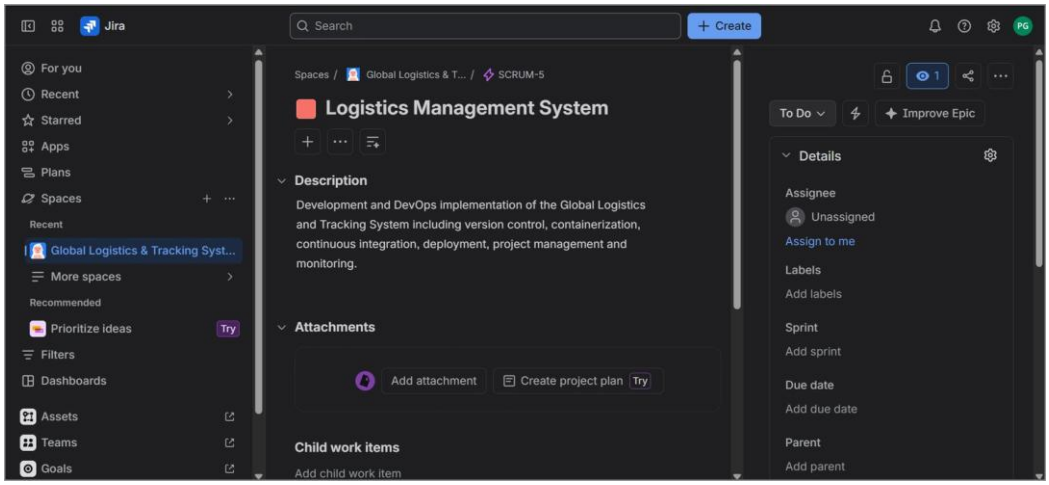


Figure 2.1: Epic SCRUM-5, Logistics Management System

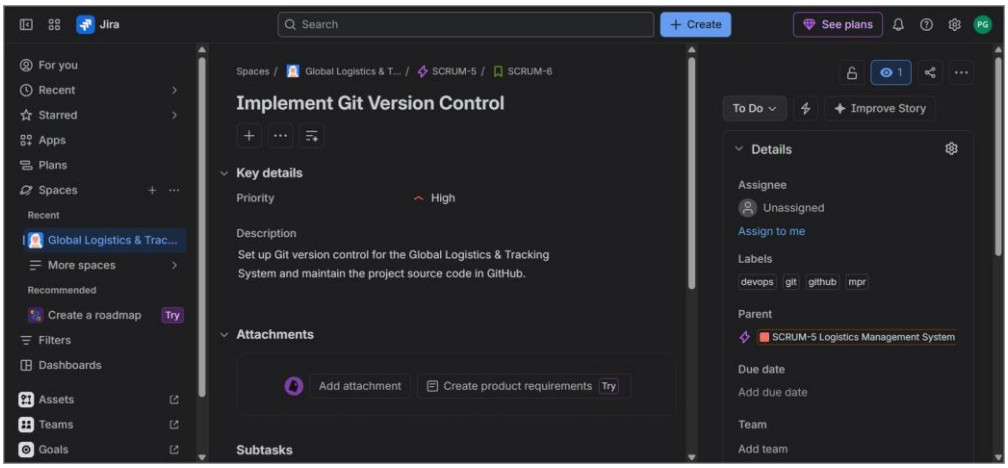


Figure 2.2: Story SCRUM-6: Implement Git Version Control

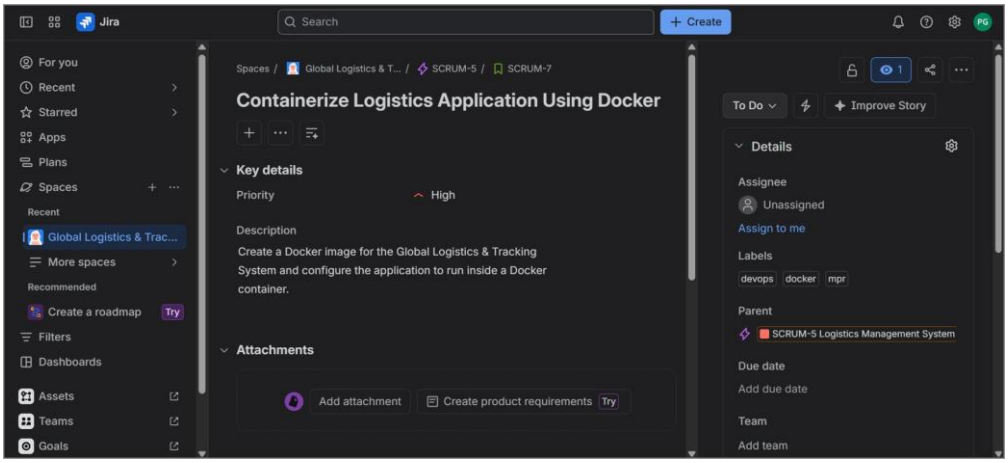


Figure 2.3: Story SCRUM-7: Containerize Logistics Application Using Docker

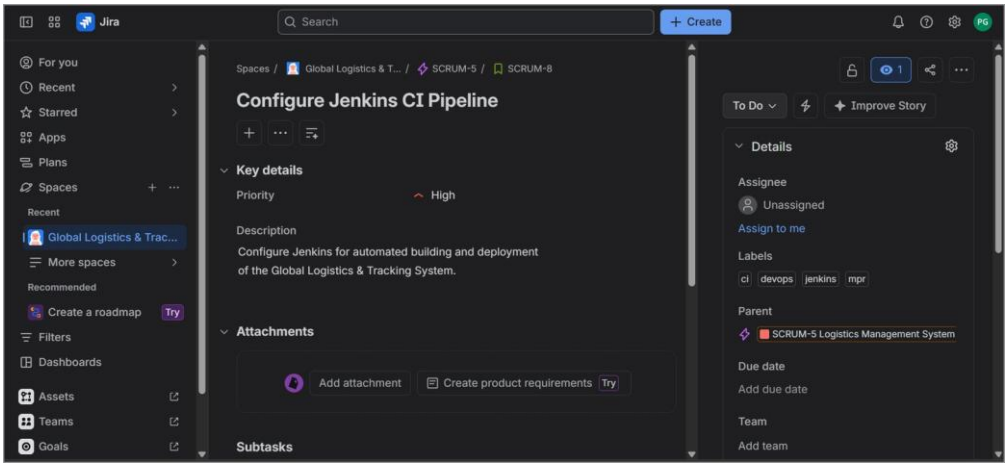


Figure 2.4: Story SCRUM-8: Configure Jenkins CI Pipeline

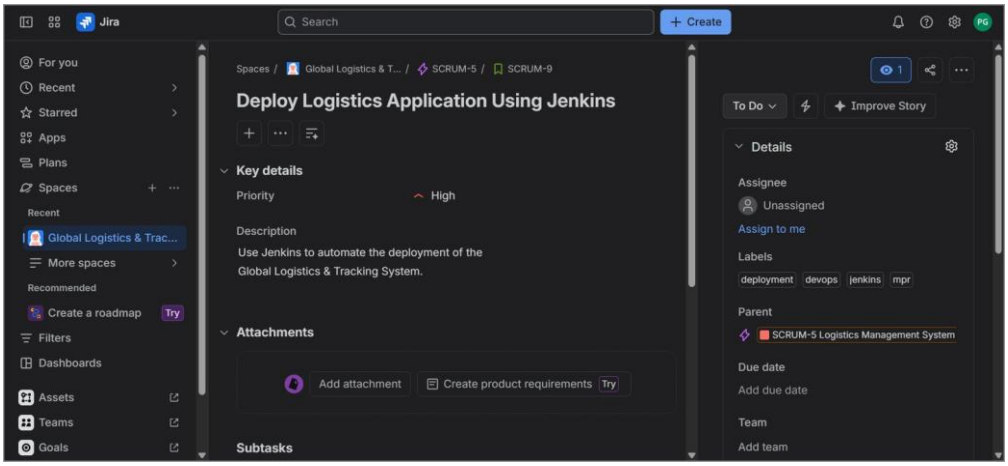


Figure 2.5: Story SCRUM-9: Deploy Logistics Application Using Jenkins

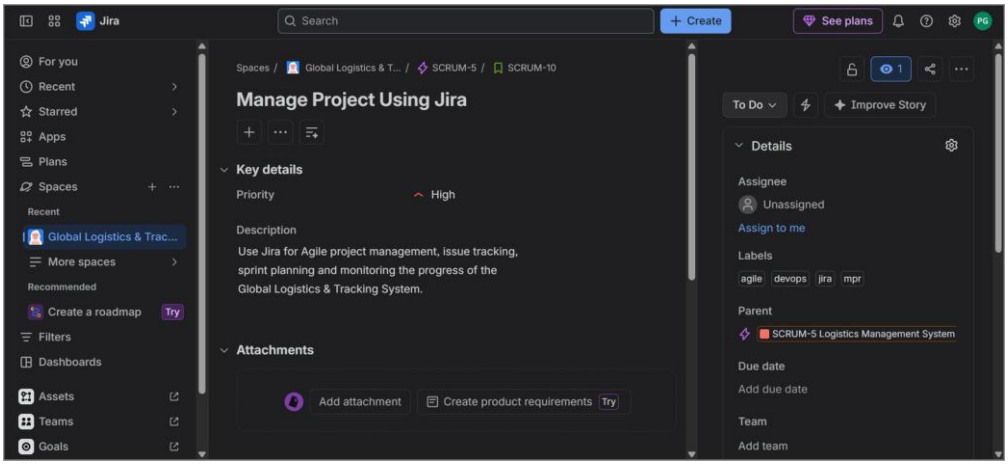


Figure 2.6: Story SCRUM-10: Manage Project Using Jira

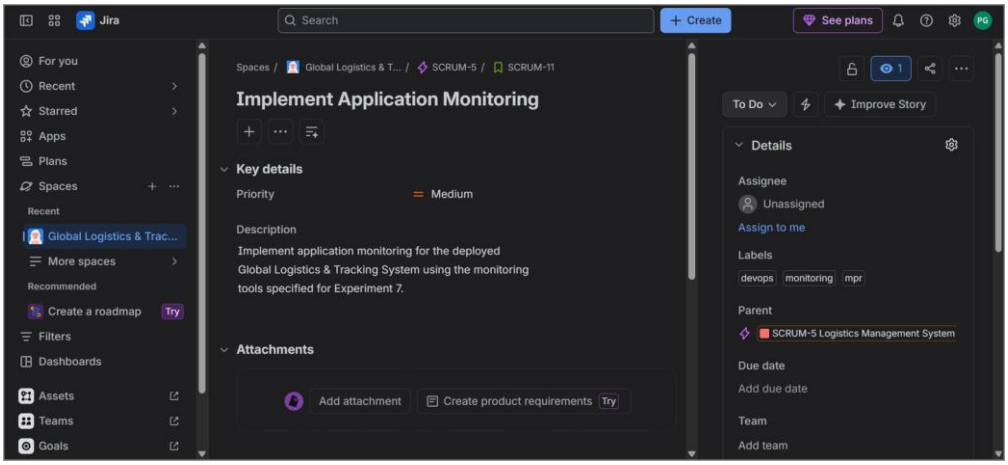


Figure 2.7: Story SCRUM-11: Implement Application Monitoring

2.6 Sprint Plan

Sprint	Goal	Scope
MPR DevOps Implementation	Take the existing Global Logistics & Tracking System from a local project to a version-controlled, containerised, automatically built and deployed and monitored system.	6 work items: SCRUM-6 to SCRUM-11

Table 2.4: Sprint goal and scope

The sprint was created in the Backlog view with all six work items (Figure 2.8), and the backlog items and their priorities are shown in Figure 2.9. The implementation work on the pipeline and the

monitoring tool took place on 4 and 5 October 2026, as shown by the timestamps in the Jenkins and Nagios screenshots in Chapter 5.

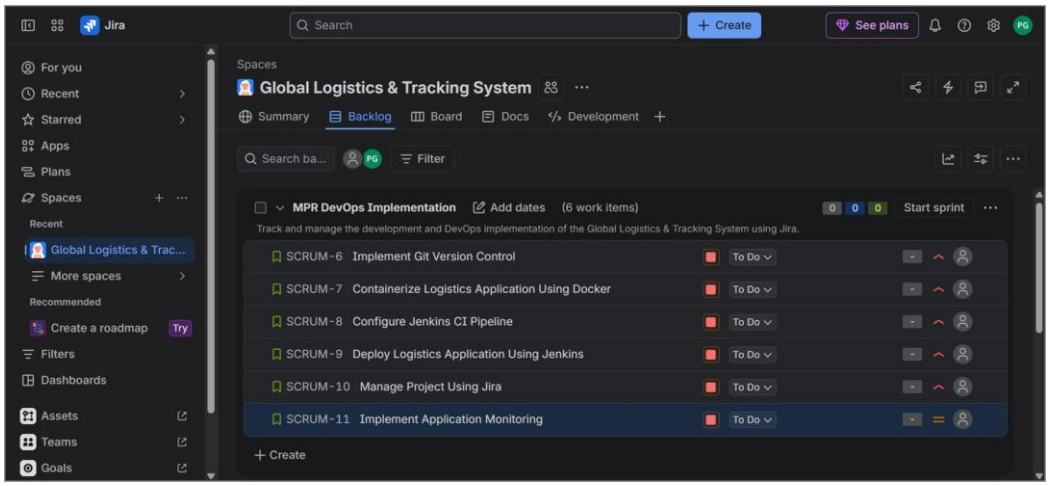


Figure 2.8: Sprint "MPR DevOps Implementation" in the Jira backlog

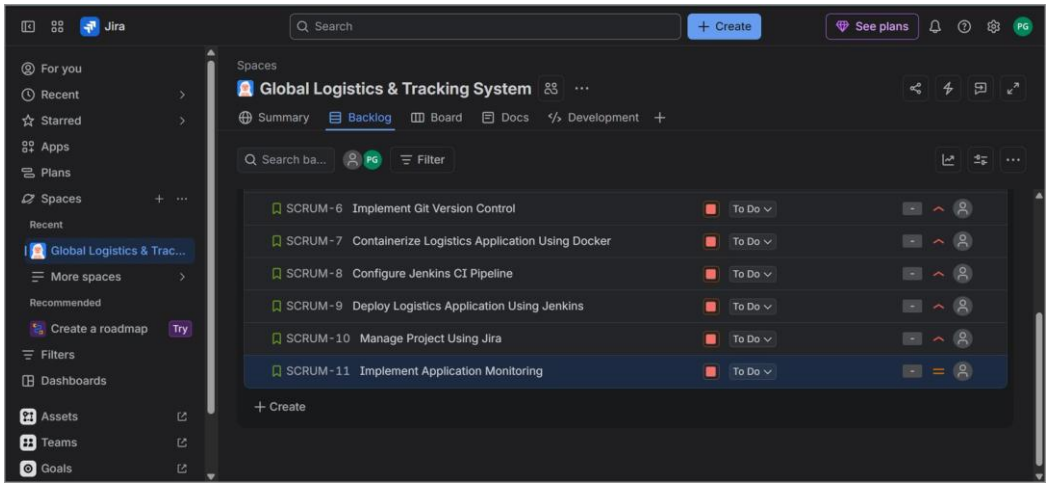


Figure 2.9: Backlog items with status and priority

2.7 Board and Progress Tracking

The Jira board has the three columns To Do, In Progress and Done. Figure 2.10 was captured while the Git and Docker stories were complete, the Jira story was in progress and monitoring was still waiting. Figure 2.11 shows the story SCRUM-10 with the project screenshots attached to it as evidence, and

Figure 2.12 shows the board at the end of the project work, with five stories in Done and the monitoring story as the remaining item at that time.

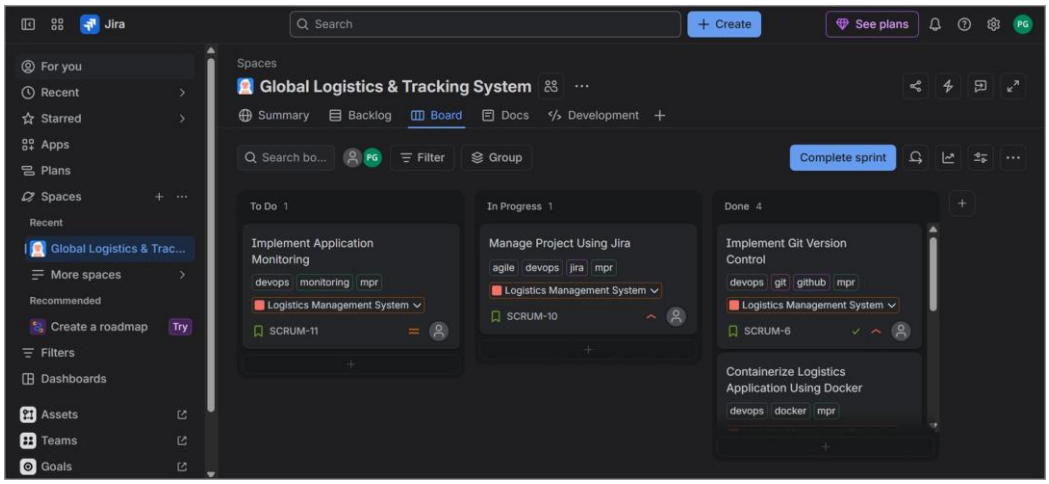


Figure 2.10: Board during the sprint

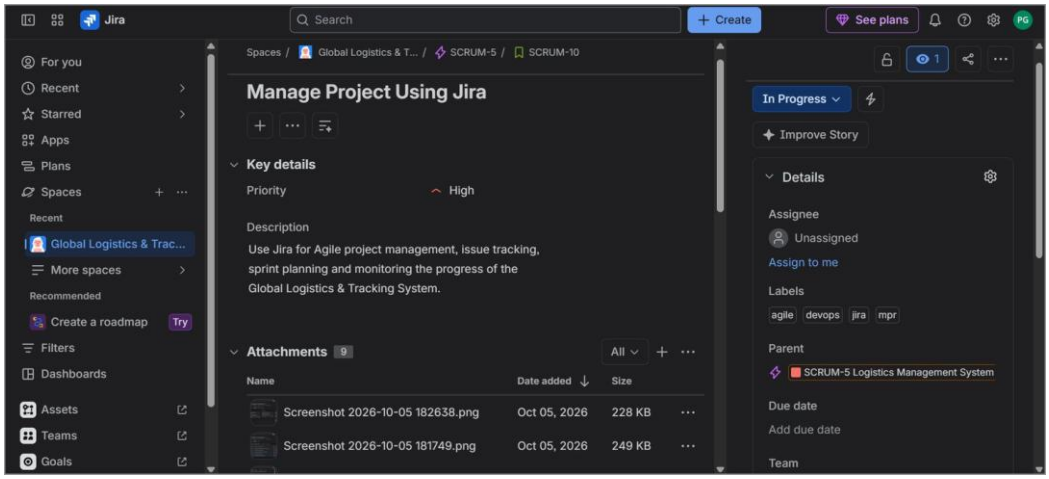


Figure 2.11: SCRUM-10 In Progress, with screenshots attached as evidence

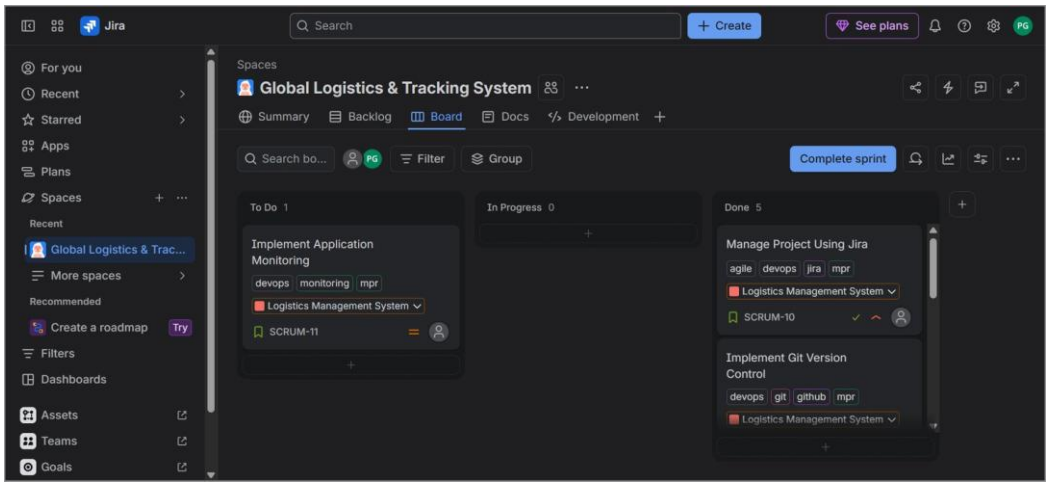


Figure 2.12: Board near the end of the sprint: five stories in Done

2.8 Definition of Done

A story was treated as done only when all of the following conditions were met.

1. The change is committed to Git and pushed to the GitHub repository.

2. Where the story involves the pipeline, the Jenkins build that contains the change finished with the status SUCCESS.
3. The application or tool behaves as expected when run (in Docker for Experiments 2 to 5, and in the Nagios web interface for Experiment 7).
4. Screenshot evidence of the result is captured and recorded, and the Jira issue is moved to Done.

3. System Design

3.1 Architecture

The system is a three-part web application packaged as containers (Figure 3.1). The browser loads the single-page interface from a Node.js and Express server that listens on port 3000. The same server exposes the data operations used by the interface and stores customers, shipments and tracking events in a MySQL database. In Docker Compose the two run as separate services, `app` and `mysql`, on one private network, and the application reaches the database by the service name `mysql` instead of `localhost`. A third container, Nagios Core, joins the same network and checks the application over HTTP and the database over TCP. Outside the stack, the developer pushes code to GitHub, and Jenkins checks the repository out, builds the images and runs Docker Compose on the host. The work itself is planned and tracked in Jira.

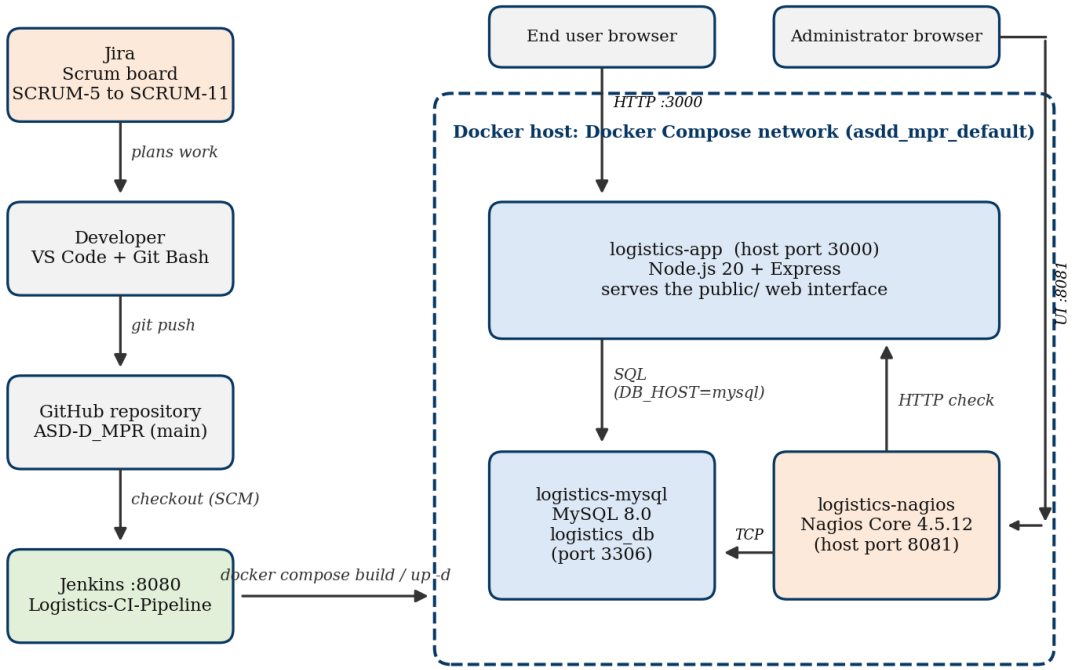


Figure 3.1: System and delivery architecture

3.2 Technology Stack

Layer	Technology	Role in the project
Frontend	HTML5, CSS3, JavaScript (ES6+), Google Fonts (Inter), Lucide icons	Single-page interface with white and blue design
Backend	Node.js 20, Express.js	Application server and data operations
Database	MySQL 8.0	Relational storage with primary and foreign keys
Libraries	dotenv, cors, mysql2	Environment configuration, cross-origin access, database driver
Version control	Git, GitHub	Source management and publication
Containers	Docker 28, Docker Compose	Image build and multi-container run
CI/CD	Jenkins 2.568	Declarative pipeline from SCM

Layer	Technology	Role in the project
Project management	Jira (Scrum project)	Epic, stories, sprint, board
Monitoring	Nagios Core 4.5.12	HTTP and TCP service checks

Table 3.1: Technology stack

3.3 Database Design

The database `logistics_db` contains ten tables, created from `database/schema.sql` (Figure 3.2): Aeroplane, Customer, Location, Ship, Shipment, Shipment_Location, Shipment_Vehicle, Tracking_Event, Truck and Vehicle. The tables cover the customers, the shipments, the locations and vehicles involved in moving them (Truck, Ship and Aeroplane alongside the general Vehicle table) and the link tables Shipment_Location and Shipment_Vehicle. The Tracking_Event table holds the timeline that the Track Shipment view displays for a given shipment. The script defines primary and foreign key constraints and inserts sample data, so that the application can be tested immediately after the database starts.

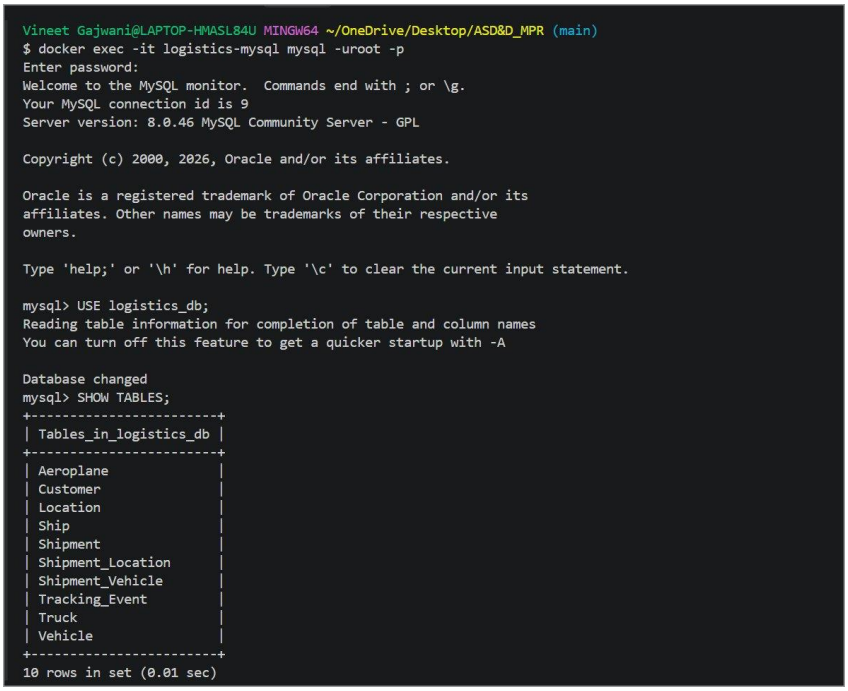


Figure 3.2: Database tables listed with SHOW TABLES inside the MySQL container

3.4 Configuration and Security Design

- Database host, user, password, name and the server port are read from environment variables (`DB_HOST`, `DB_USER`, `DB_PASSWORD`, `DB_NAME`, `PORT`) through dotenv, so that the same code runs on the developer machine (host `localhost`) and in Docker Compose (host `mysql`).
- A `.dockerignore` file excludes local files such as the `.env` file and `node_modules` from the Docker build context, so that credentials are not baked into the image.
- Only the application port 3000 and the Nagios port 8081 are intended for the user; the database port is reached inside the Compose network.

4. Application Implementation

The application was developed before the lab experiments began and was not rebuilt for them; the DevOps work was added around it. This chapter describes the application that was delivered.

4.1 Project Structure

The project folder (Figure 4.1) contains `server.js` (the Express server), `package.json` and `package-lock.json` (dependencies), `database/schema.sql` (the MySQL schema and sample data), `public/` (the web interface: HTML, CSS, JavaScript and image assets) and `README.md`. The Dockerfile, `.dockerignore`, `docker-compose.yml` and `Jenkinsfile` were added during Experiments 2 to 5.

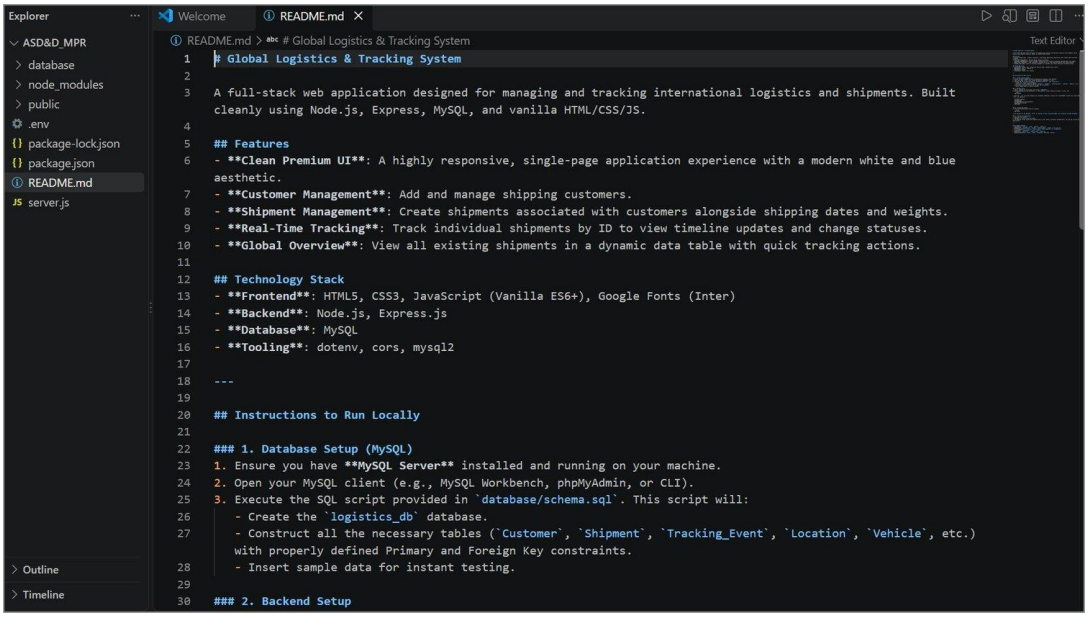


Figure 4.1: Project in Visual Studio Code with the README open

4.2 Features

- **Dashboard.** A landing view with a summary hero banner and the quick actions Track Order Now and Ship Faster.
- **Customer management.** The Add Customer view registers a new customer.
- **Shipment management.** The Create Shipment view creates a shipment associated with a customer, with its shipping date and weight.
- **Real-time tracking.** The Track Shipment view looks up a shipment by its identifier and shows the timeline of tracking events and status changes.
- **Global overview.** The All Shipments view lists the existing shipments in a dynamic data table with quick tracking actions.

4.3 User Interface

The interface uses a modern white and blue design with a left navigation bar (LogistiQ, Dashboard, Add Customer, Create Shipment, Track Shipment and All Shipments) and a help card. Figure 4.2 shows the dashboard of the application running in Docker.

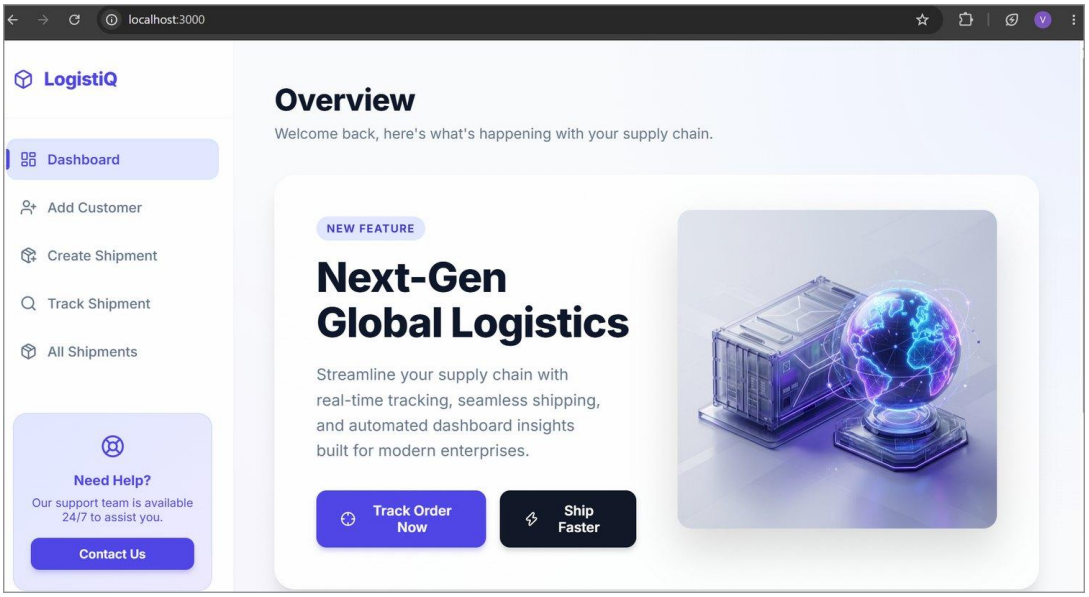


Figure 4.2: LogistiQ dashboard served from the Docker container at localhost:3000

4.4 Server and Database Connection

The server starts on the configured port and then opens a connection to MySQL using the credentials from the environment. When both succeed it writes two lines to the console, *Server is running on http://localhost:3000* and *Connected to MySQL Database!*, which are used in the Docker experiments as the signal that the container is healthy. If the database is not yet ready the connection is refused and an error is logged; Section 5.3 shows how this appeared during Compose start-up.

5. DevOps Implementation

This chapter describes how each lab experiment was applied to the project. Figure 5.1 summarises the delivery flow that connects them: a planned story leads to a code change, which is committed and pushed to GitHub, built and validated by Jenkins, deployed with Docker Compose and then observed by Nagios; defects found there return to the backlog.

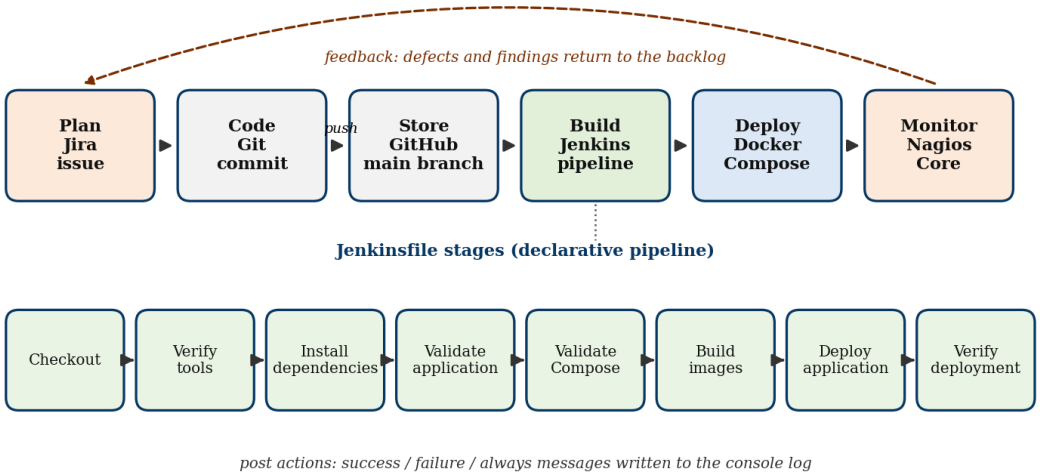


Figure 5.1: Delivery flow from planning to monitoring

5.1 Version Control with Git and GitHub (Experiment 1)

The existing project was placed under Git. A repository named *ASD-D_MPR* was created on GitHub without any initial files, the local project was initialised as a Git repository, all files were staged and committed, and the main branch was pushed to the remote. The Jira story SCRUM-6 corresponds to this experiment.

```
git init
git add .
git commit -m "Global Logistics and Tracking"
git push https://github.com/Vineet-B-Gajwani/ASD-D_MPR main
```

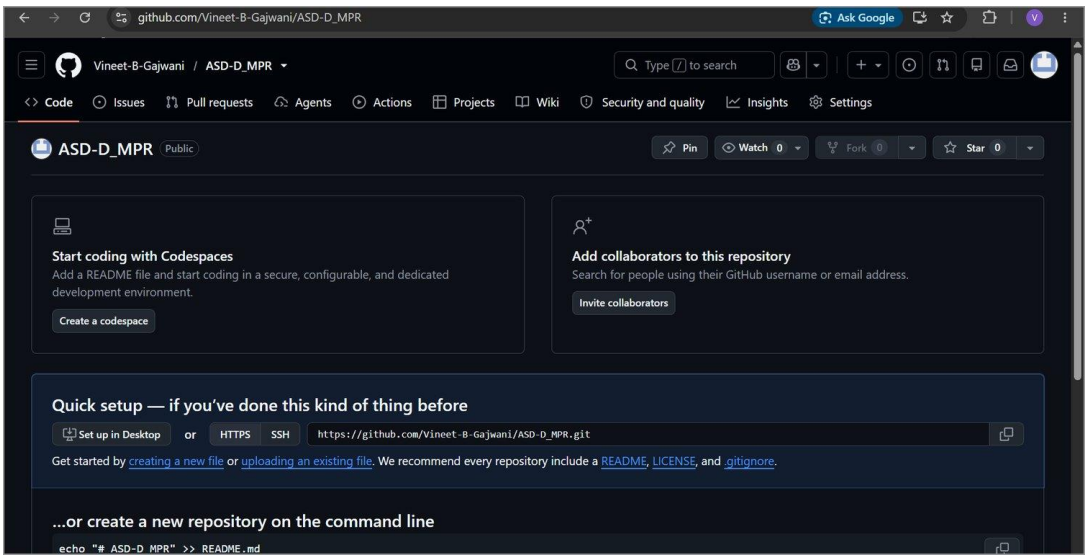


Figure 5.2: New GitHub repository ASD-D_MPR created (quick setup page)



Figure 5.3: Staging all files with git add

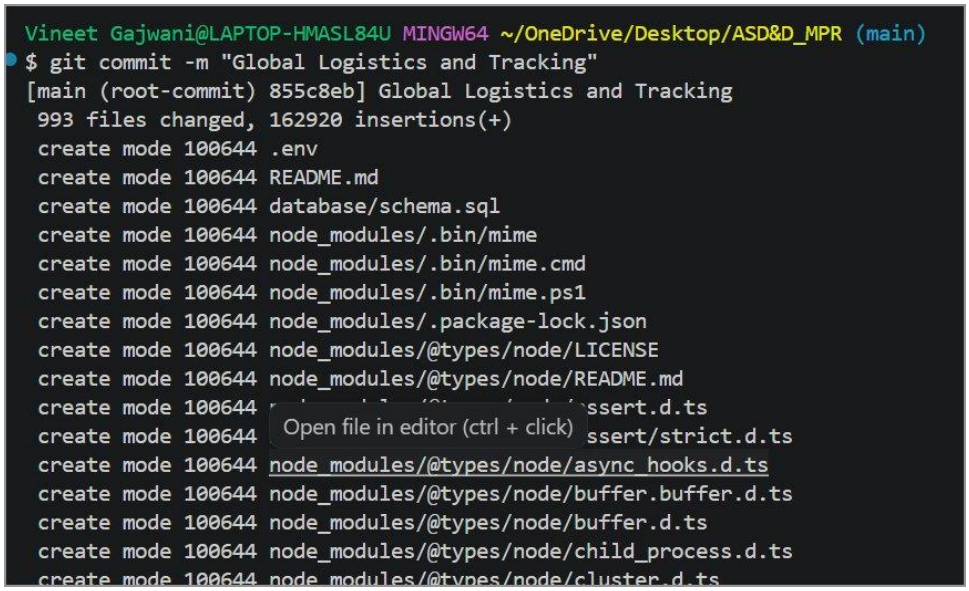


Figure 5.4: Initial commit on the main branch (root-commit 855c8eb)

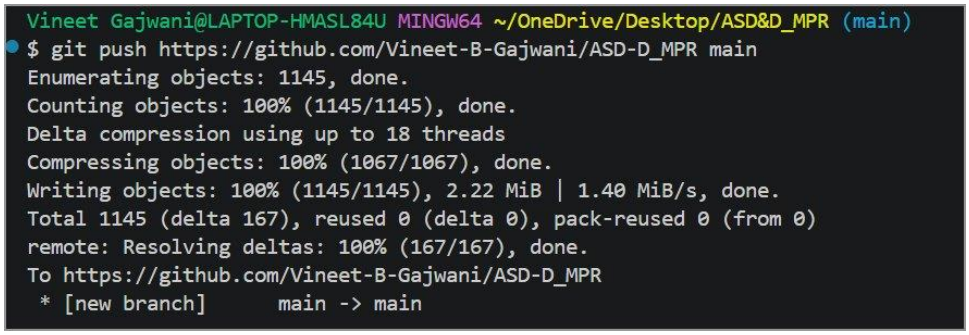


Figure 5.5: Pushing the main branch to GitHub

The push created the branch *main* on the remote. Later increments were committed with descriptive messages, which the GitHub file list shows (Figure 5.32 in Section 5.5): *Global Logistics and Tracking*, *Done till exp 3*, *Add automated application deployment* and *Update Logistics application frontend*. These messages mark the end of each experiment and make the history readable as a record of the sprint.

5.2 Containerisation with Docker (Experiment 2)

The Node.js application was containerised with a Dockerfile in the project root. The image uses Node 20 as its base, copies the package files first and runs `npm install` so that the dependency layer is cached whenever only source code changes, then copies the rest of the project and starts the server with `npm start`. A `.dockerignore` file keeps local files out of the build context.

```
FROM node:20
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 3000
CMD ["npm", "start"]
```

Docker was first verified (Figure 5.6), then the image was built with `docker build -t logistics-app .`. The first build took 176 seconds, because the Node base image had to be pulled; the resulting image is listed with its size in Figure 5.8. The container was started in detached mode with port 3000 published, and its logs confirmed that the server was running and had connected to MySQL.

```
Vineet Gajwani@LAPTOP-HMASL84U MINGW64 ~/OneDrive/Desktop/ASD&D_MPR (main)
$ docker --version
Docker version 28.5.1, build e180ab8

Vineet Gajwani@LAPTOP-HMASL84U MINGW64 ~/OneDrive/Desktop/ASD&D_MPR (main)
$ docker info
Client:
 Version:    28.5.1
 Context:    desktop-linux
 Debug Mode: false
 Plugins:
  ai: Docker AI Agent - Ask Gordon (Docker Inc.)
    Version:  v1.9.11
    Path:     C:\Program Files\Docker\cli-plugins\docker-ai.exe
  buildx: Docker Buildx (Docker Inc.)
    Version:  v0.29.1-desktop.1
    Path:     C:\Program Files\Docker\cli-plugins\docker-buildx.exe
  cloud: Docker Cloud (Docker Inc.)
    Version:  v0.4.39
    Path:     C:\Program Files\Docker\cli-plugins\docker-cloud.exe
  compose: Docker Compose (Docker Inc.)
    Version:  v2.40.0-desktop.1
```

Figure 5.6: Docker version and installation details

```
Vineet Gajwani@LAPTOP-HMASL84U MINGW64 ~/OneDrive/Desktop/ASD&D_MPR (main)
$ docker build -t logistics-app .
[+] Building 176.0s (11/11) FINISHED                                                                    docker:desktop-linux
 => [internal] load build definition from Dockerfile                                                                0.1s
 => => transferring dockerfile: 160B                                                                                0.0s
 => [internal] load metadata for docker.io/library/node:20                                                          3.9s
 => [auth] library/node:pull token for registry-1.docker.io                                                         0.0s
 => [internal] load .dockerignore                                                                                   0.1s
 => => transferring context: 91B                                                                                    0.0s
 => [1/5] FROM docker.io/library/node:20@sha256:8f693eaa7e0a8e71560c9a82b55fd54c2ae920a2ba5d2cde28bac7d1c01c9ba5  166.7s
 => => resolve docker.io/library/node:20@sha256:8f693eaa7e0a8e71560c9a82b55fd54c2ae920a2ba5d2cde28bac7d1c01c9ba5    0.1s
 => => sha256:0b22ec2d9b1a1a0e906e301b5ef08efe7774d5fc001b9cfa08e4e980152a0357 448B / 448B                          0.4s
 => => sha256:2f2ab6e4eb4619f7b8506c38dc79c8adf77acffb7622bc15ceeabb0fbc73b8a1 1.25MB / 1.25MB                     56.9s
 => => sha256:76e6b8204a89d20e7915fdc9a98ff33cb65fd7ce44ecf5a84b6cfca4bdb3d6d0 48.62MB / 48.62MB                  127.5s
 => => sha256:c83419f4430962505175b565eb8a24aafeaa4d441c0b22234e94c6e8160848ba 3.32kB / 3.32kB                      3.2s
 => => sha256:2933ecab0f11302fd71f29aa83ad2904683246f7a8320ad0dc3a60b23f05fee9 211.55MB / 211.55MB                156.7s
 => => sha256:8fcda2b4d7993820b00c5488d173051e76d01ba6b85620617ba77001b0f9e2fa 64.40MB / 64.40MB                  123.3s
 => => sha256:e5203b2bfeff92b72e816dc6cbb1f16856f0cd592e521e3c0cfa195a78fe180e 24.04MB / 24.04MB                   49.5s
 => => sha256:db53381ee51f9e43304e236099ba097ae1b33ae41a8007e0b6319992eb55fd00 48.49MB / 48.49MB                   29.2s
 => => extracting sha256:db53381ee51f9e43304e236099ba097ae1b33ae41a8007e0b6319992eb55fd00                           2.2s
 => => extracting sha256:e5203b2bfeff92b72e816dc6cbb1f16856f0cd592e521e3c0cfa195a78fe180e                           0.8s
 => => extracting sha256:8fcda2b4d7993820b00c5488d173051e76d01ba6b85620617ba77001b0f9e2fa                           2.9s
 => => extracting sha256:2933ecab0f11302fd71f29aa83ad2904683246f7a8320ad0dc3a60b23f05fee9                           6.6s
 => => extracting sha256:c83419f4430962505175b565eb8a24aafeaa4d441c0b22234e94c6e8160848ba                           0.0s
 => => extracting sha256:76e6b8204a89d20e7915fdc9a98ff33cb65fd7ce44ecf5a84b6cfca4bdb3d6d0                           2.5s
 => => extracting sha256:2f2ab6e4eb4619f7b8506c38dc79c8adf77acffb7622bc15ceeabb0fbc73b8a1                           0.1s
 => => extracting sha256:0b22ec2d9b1a1a0e906e301b5ef08efe7774d5fc001b9cfa08e4e980152a0357                           0.0s
 => [internal] load build context                                                                                   0.3s
```

Figure 5.7: Building the image: docker build -t logistics-app .

```
Vineet Gajwani@LAPTOP-HMASL84U MINGW64 ~/OneDrive/Desktop/ASD&D_MPR (main)
$ docker images
REPOSITORY           TAG       IMAGE ID       CREATED          SIZE
logistics-app        latest    8111dc24aa2a   5 minutes ago    1.6GB
patient-management   latest    778a8a870020   3 weeks ago      225MB
```

Figure 5.8: The logistics-app image listed by docker images (1.6 GB)

```
Vineet Gajwani@LAPTOP-HMASL84U MINGW64 ~/OneDrive/Desktop/ASD&D_MPR (main)
$ docker run -d -p 3000:3000 --name logistics-container logistics-app
9ac4bbc7d45ce02249199a5c28562d4ee867a70be0cedf6aaeb240d1c6e4b434
```

Figure 5.9: Starting the container: docker run -d -p 3000:3000 --name logistics-container logistics-app

```
Vineet Gajwani@LAPTOP-HMASL84U MINGW64 ~/OneDrive/Desktop/ASD&D_MPR (main)
$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED          STATUS          PORTS                                                  NAMES
9ac4bbc7d45c   logistics-app  "docker-entrypoint.s…"   44 seconds ago   Up 42 seconds   0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp   logistics-container
```

Figure 5.10: The running container listed by docker ps

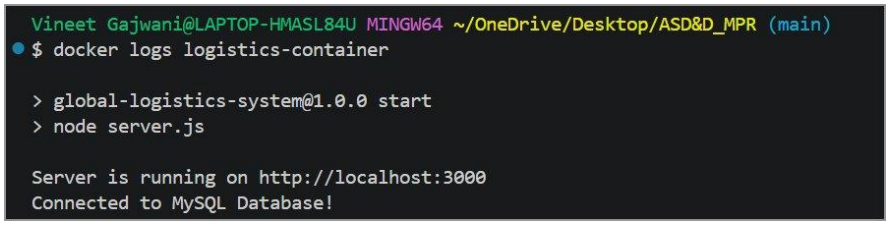


Figure 5.11: Container logs: server running and connected to the MySQL database

5.3 Multi-Container Orchestration with Docker Compose (Experiment 3)

The application needs a database, and running MySQL by hand beside the application container does not give a repeatable environment. A `docker-compose.yml` file therefore defines the whole system as two services on one network, so that a single command builds the image and starts both containers.

Service / container	Image	Ports	Notes
app (logistics-app)	asdd_mpr-app, built from the project Dockerfile	3000 to 3000	Node.js and Express application; connects to the database by the host name `mysql`
mysql (logistics-mysql)	mysql:8.0	3306 and 33060 inside the network	Holds `logistics_db`; the schema is available as `database/schema.sql`
Network	asdd_mpr_default	—	Created automatically by Compose; both containers are attached to it

Table 5.1: Services in the Compose stack

The stack is started with `docker compose up --build`. Figure 5.12 shows the build, the creation of the network and of both containers; Figure 5.13 shows both containers running. The application dashboard served by this stack is shown in Figure 4.2.

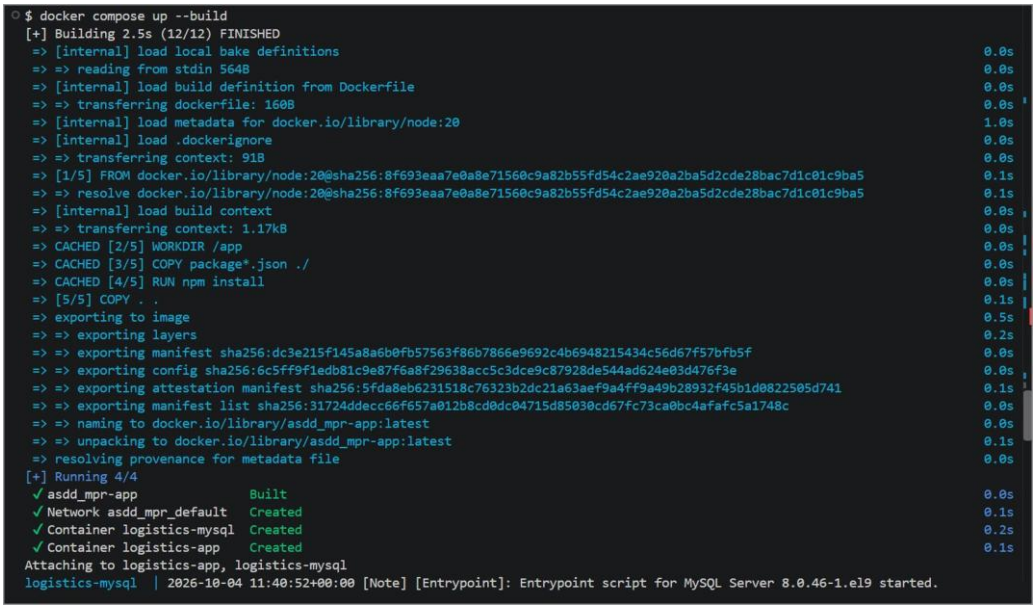


Figure 5.12: docker compose up --build: image built, network and two containers created

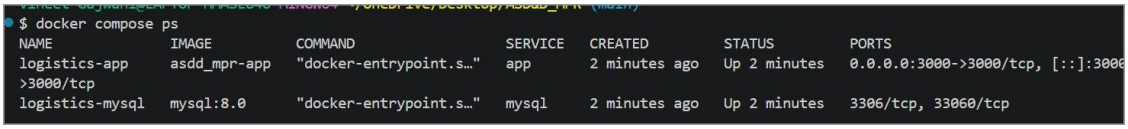


Figure 5.13: docker compose ps: logistics-app and logistics-mysql running

Start-up ordering. The application container logged *ECONNREFUSED* for the address of the MySQL container (172.18.0.2:3306) while the database was still initialising, and after the database became ready the server connected and printed *Connected to MySQL Database!* (Figure 5.14). The log also confirms that the host name `mysql` was resolved by the Compose network, which is the central point of this experiment: inside Docker the application must reach the database by service name, not by `localhost`.

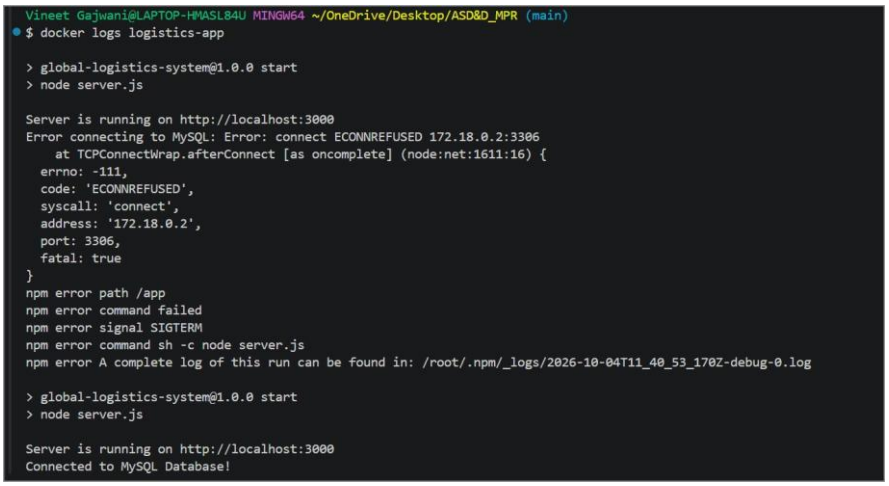


Figure 5.14: Application logs: connection refused during database start-up, then connected

Finally the database content was verified from inside the MySQL container: `docker exec -it logistics-mysql mysql -uroot -p` opened the MySQL client, `USE logistics_db;` selected the database and `SHOW TABLES;` listed the ten tables shown earlier in Figure 3.2.

5.4 Continuous Integration with Jenkins (Experiment 4)

Jenkins 2.568.2 was run on the developer machine and a Pipeline job named *Logistics-CI-Pipeline* was created. Before writing the pipeline, the tools it depends on were checked on the Jenkins host: the Docker engine and Docker Compose version, the Node.js syntax check and `npm ci`, validation of the Compose file with `docker compose config --quiet` and the image build (Figures 5.15 and 5.16).

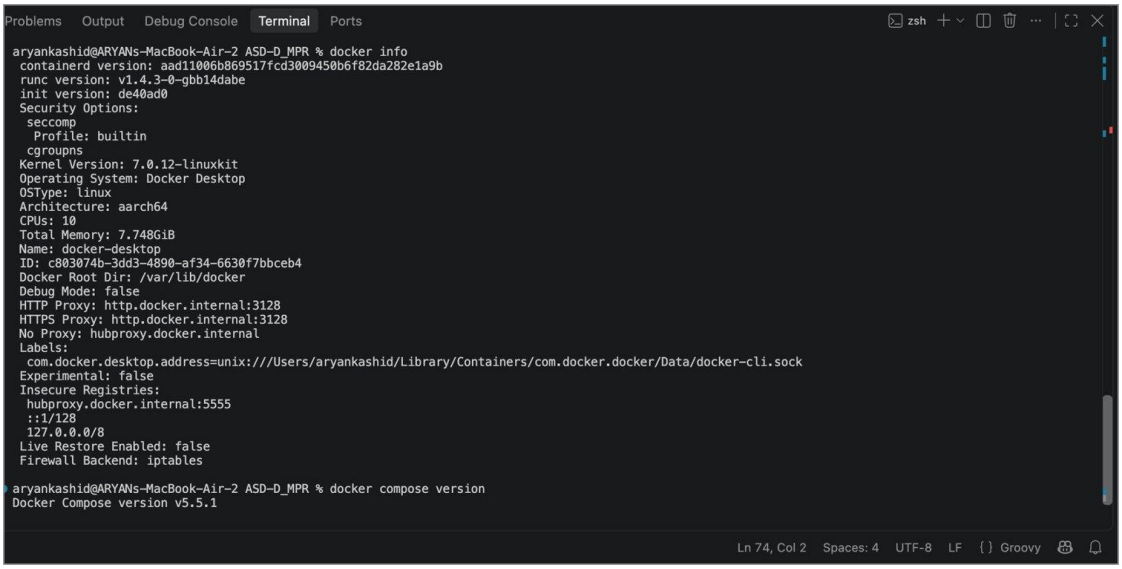


Figure 5.15: Docker engine information and Docker Compose version on the Jenkins host

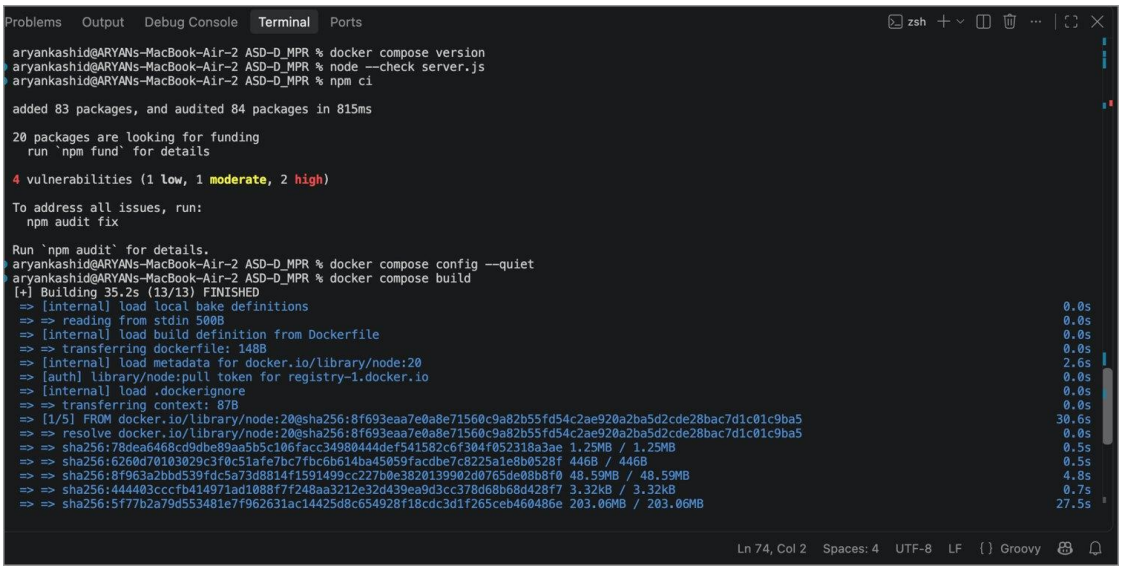


Figure 5.16: Dependency installation, Compose validation and image build on the Jenkins host

The pipeline is declarative and is stored in the repository as `Jenkinsfile`, so it is versioned with the code. It runs on any agent, sets the `PATH` explicitly so that Docker, Node.js and Git are found when Jenkins runs as a background service, disables concurrent builds and adds timestamps to the console. Table 5.2 lists its stages.

Stage	What it does	Purpose
Checkout	`checkout scm` fetches the project from GitHub	Build exactly what is in the repository
Verify Development Tools	Prints versions of Java, Git, Node.js, npm, Docker CLI, Docker Compose and checks the Docker engine with `docker info`	Fail early if the environment is incomplete
Install Dependencies	Installs the Node.js dependencies with npm	Reproducible dependencies
Validate Application	`node --check server.js`	Catch syntax errors before building

Stage	What it does	Purpose
Validate Docker Compose	`docker compose config --quiet`	Catch configuration errors in the stack definition
Build Docker Images	`docker compose build`	Produce the application image
Deploy Application *(Exp 5)*	`docker compose up -d`	Start the stack from the new image
Verify Deployment *(Exp 5)*	`docker compose ps`	Show that the containers are running
Post actions	Messages on success, failure and always	Clear result in the console

Table 5.2: Stages of the Jenkins pipeline

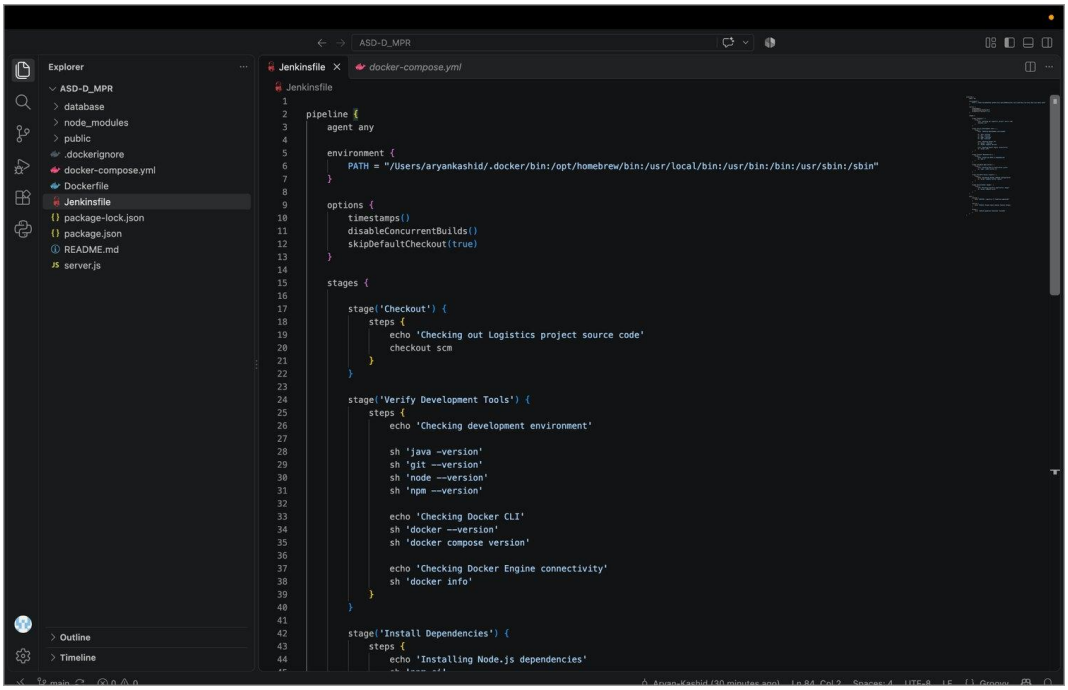


Figure 5.17: Jenkinsfile (first part): environment, options and early stages

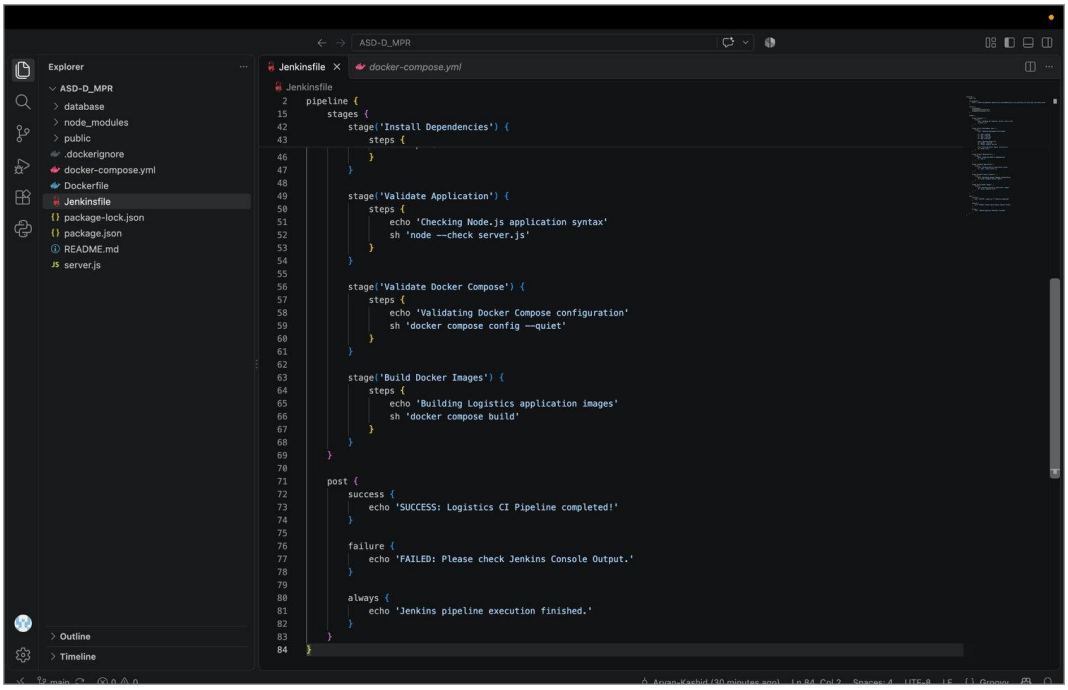


Figure 5.18: Jenkinsfile (second part): validation, build and post actions

The job was created as a Pipeline item (Figure 5.19), its definition was set to *Pipeline script from SCM* with Git and the GitHub repository URL (Figure 5.20), and the branch `*/main` and the script path `Jenkinsfile` were configured (Figure 5.21).

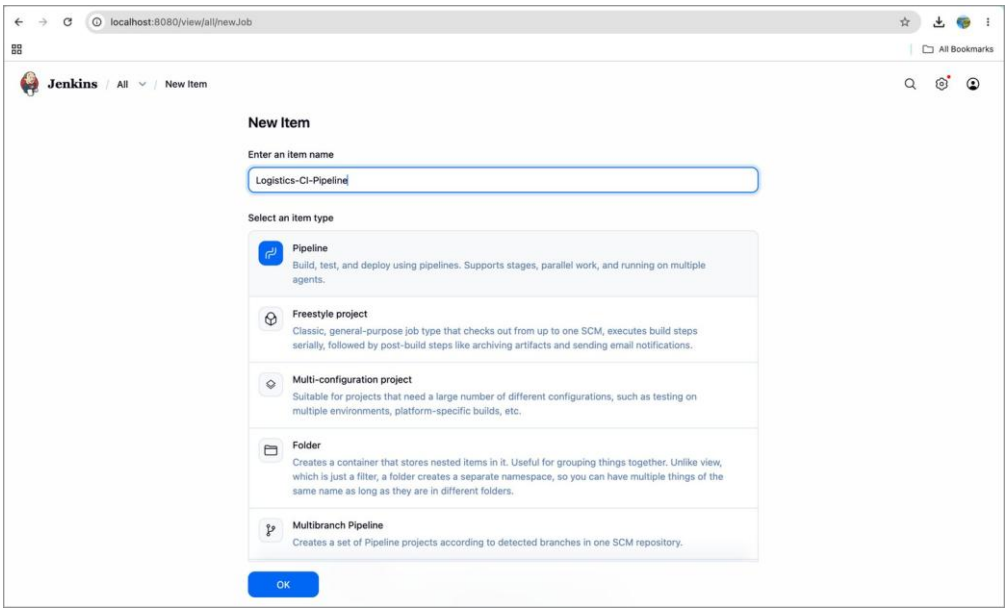


Figure 5.19: Creating the Pipeline job Logistics-CI-Pipeline

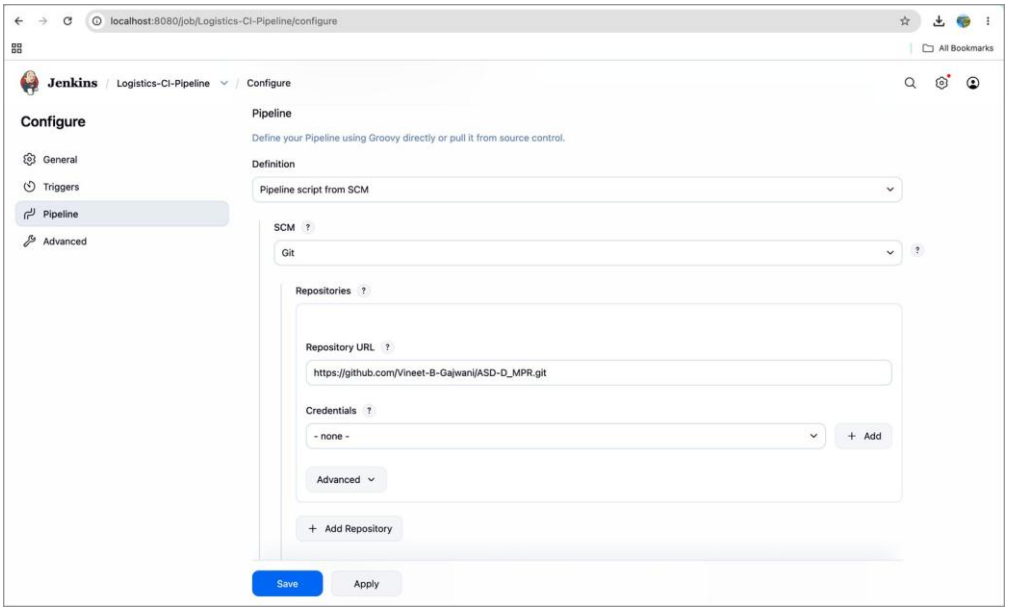


Figure 5.20: Pipeline definition: script from SCM, Git, repository URL

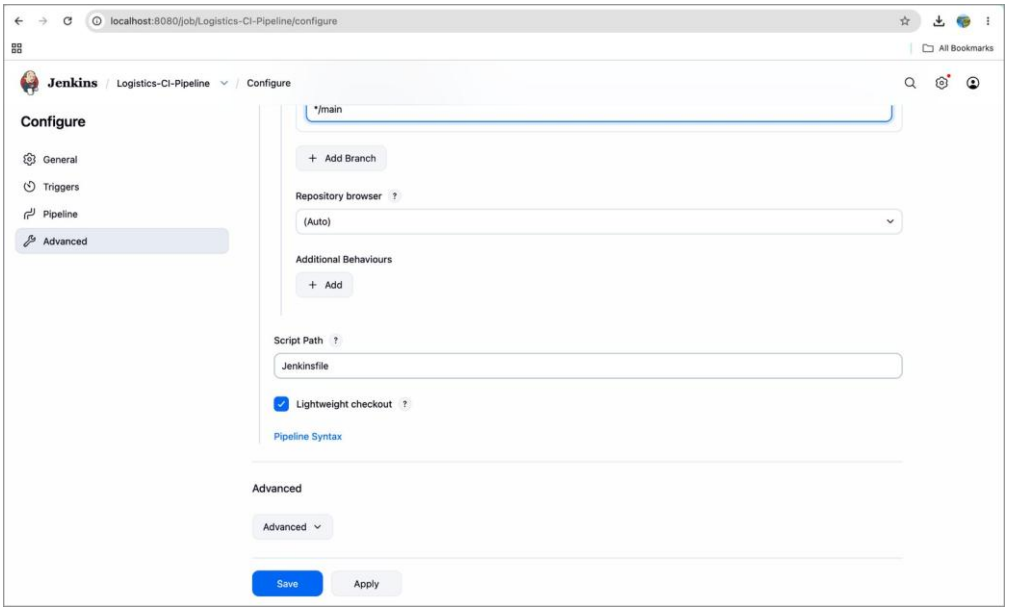


Figure 5.21: Branch to build and Jenkinsfile script path

The job status page (Figure 5.22) records the history of the pipeline. Builds 1 to 4 failed while the environment and pipeline were being configured; build 5 was the first to succeed. Its console output

(Figures 5.23 and 5.24) shows the Jenkinsfile obtained from GitHub, the source checked out, the Docker image *logistics-ci-pipeline-app* built and the final status SUCCESS.

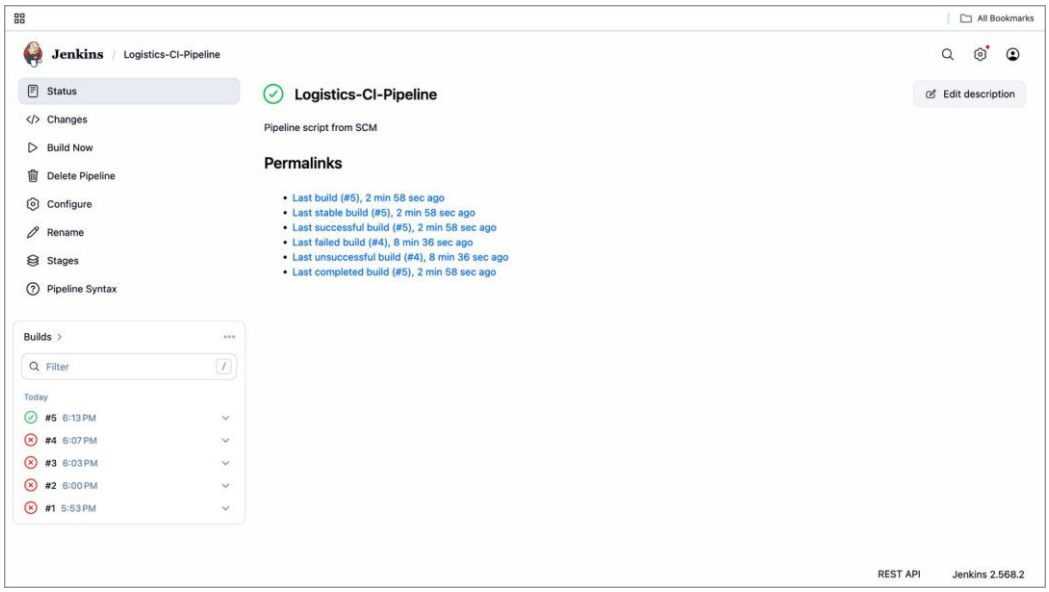


Figure 5.22: Job status: builds 1 to 4 failed, build 5 succeeded

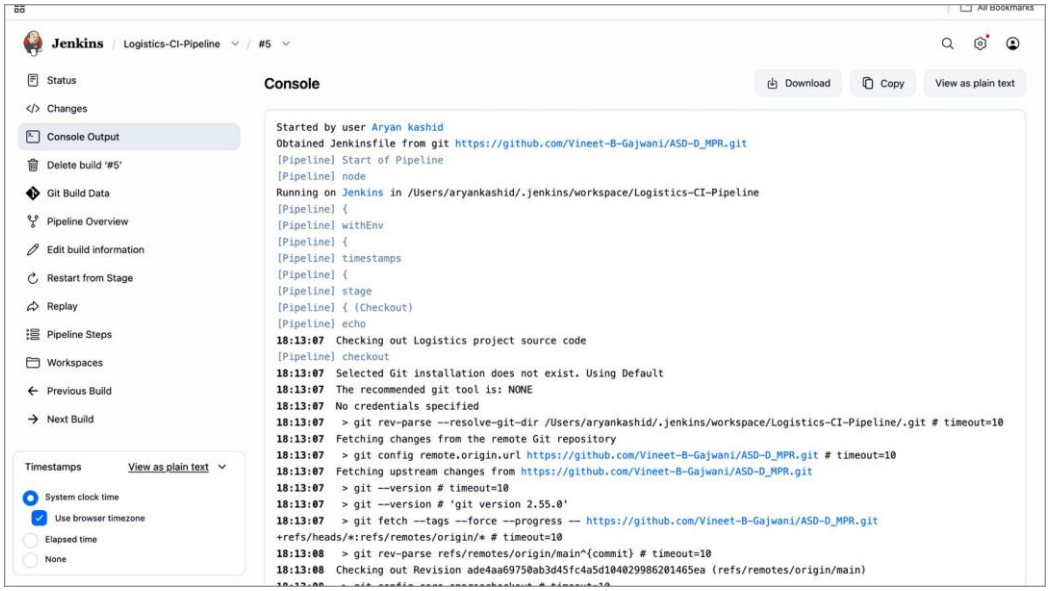


Figure 5.23: Console output of build 5: Jenkinsfile obtained from GitHub and checkout

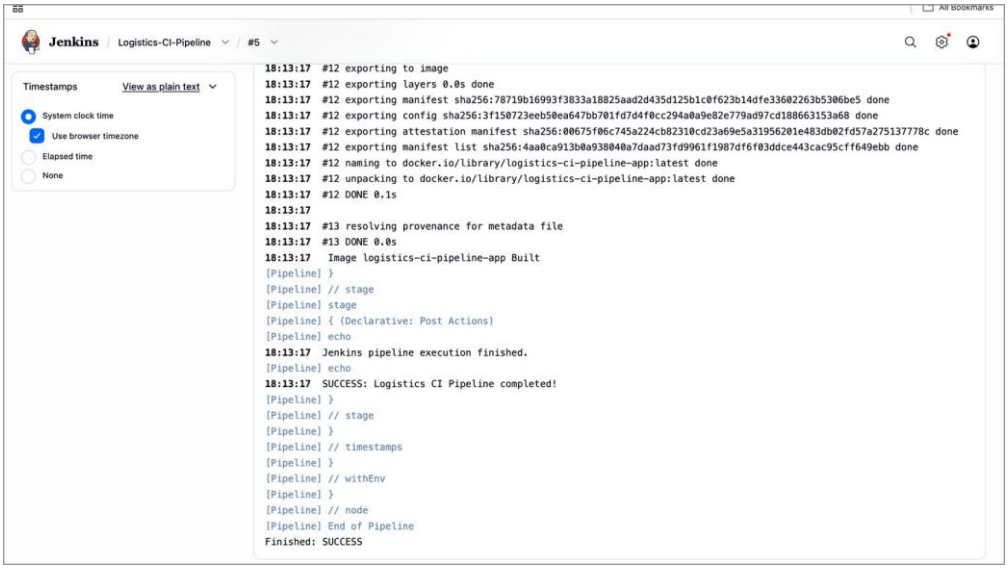


Figure 5.24: Console output of build 5: image built and Finished: SUCCESS

5.5 Automated Deployment (Experiment 5)

The pipeline was extended with two stages, *Deploy Application* and *Verify Deployment*. Deploy runs `docker compose up -d`, which recreates the containers from the freshly built image, and Verify runs `docker compose ps` to show that both containers are running. The change was committed as *Add automated application deployment* (Figure 5.32 shows it in the GitHub file list).

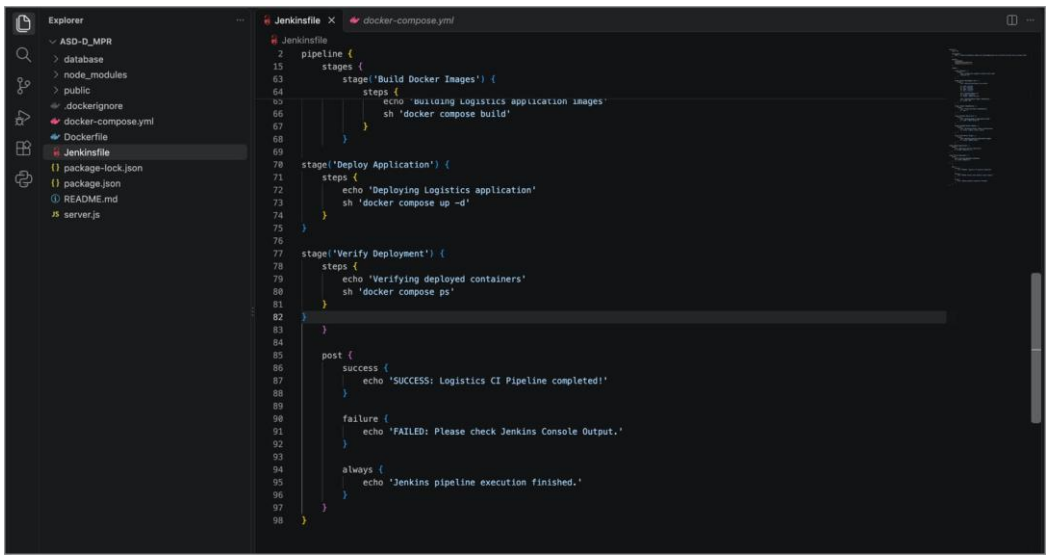


Figure 5.25: Jenkinsfile with the Deploy Application and Verify Deployment stages

Builds 6 and 7 ran the extended pipeline and both succeeded (Figure 5.26). Build 7, which took 1 minute 33 seconds, was started on 4 October 2026 at 19:41 and is linked to the commit *Add automated application deployment* (Figure 5.27). Its console output ends with `docker compose ps` showing

logistics-app and logistics-mysql up, followed by Finished: SUCCESS (Figure 5.28). The application is then reachable at `http://localhost:3000` (Figure 5.29).

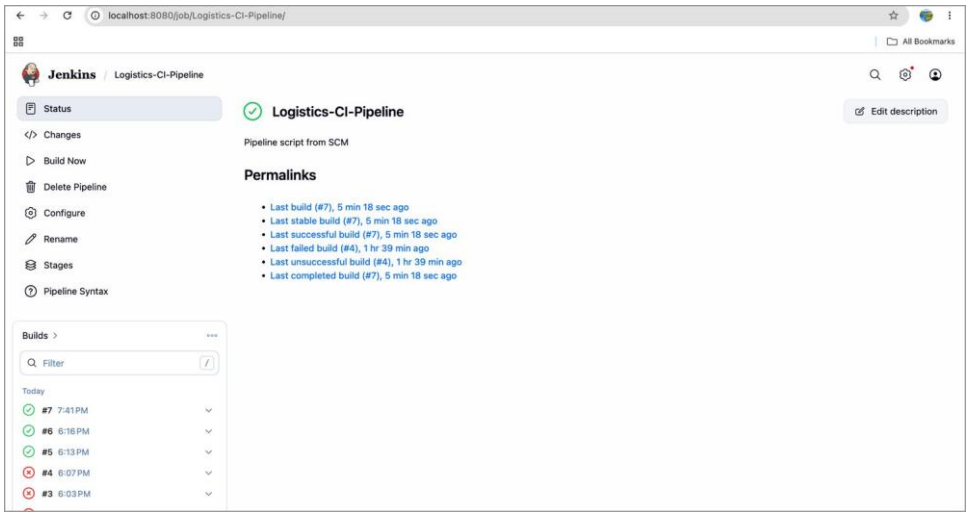


Figure 5.26: Job status: builds 5, 6 and 7 successful

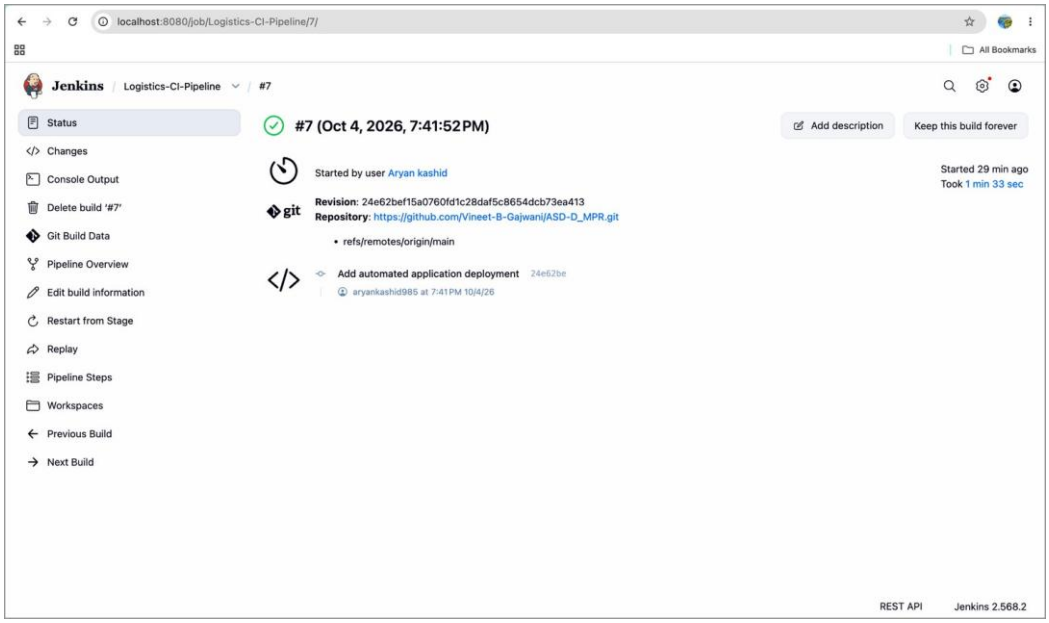


Figure 5.27: Build 7: revision and commit that triggered the deployment

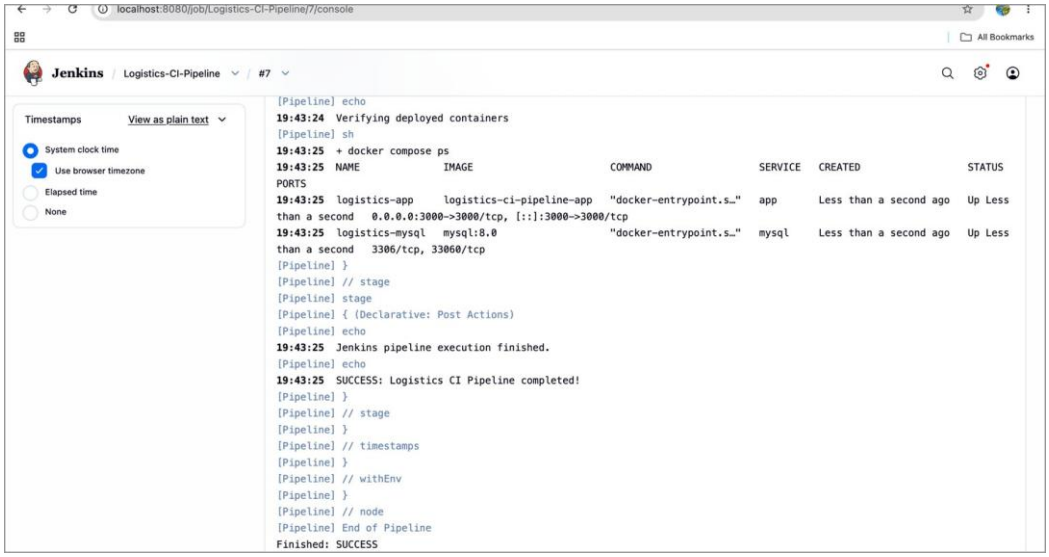


Figure 5.28: Build 7 console: containers listed as running after deployment

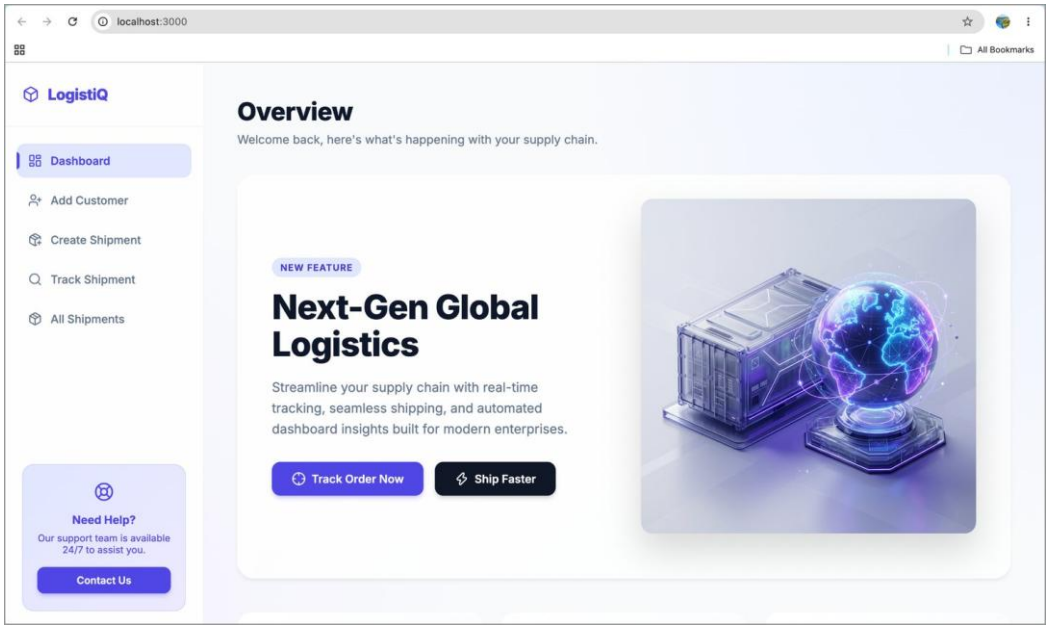


Figure 5.29: Application deployed by the pipeline (heading: Next-Gen Global Logistics)

Demonstrating push-to-deploy with a visible change

To show that a code change travels from the editor to the running container through the pipeline, the hero heading of the dashboard in `public/index.html` was changed from *Next-Gen Global Logistics* to *Current-Gen Global Logistics* (Figure 5.30). The change was staged, committed with the message

Update Logistics application frontend and pushed to GitHub (Figure 5.31), and GitHub then showed the new commit in the `public` folder (Figure 5.32).

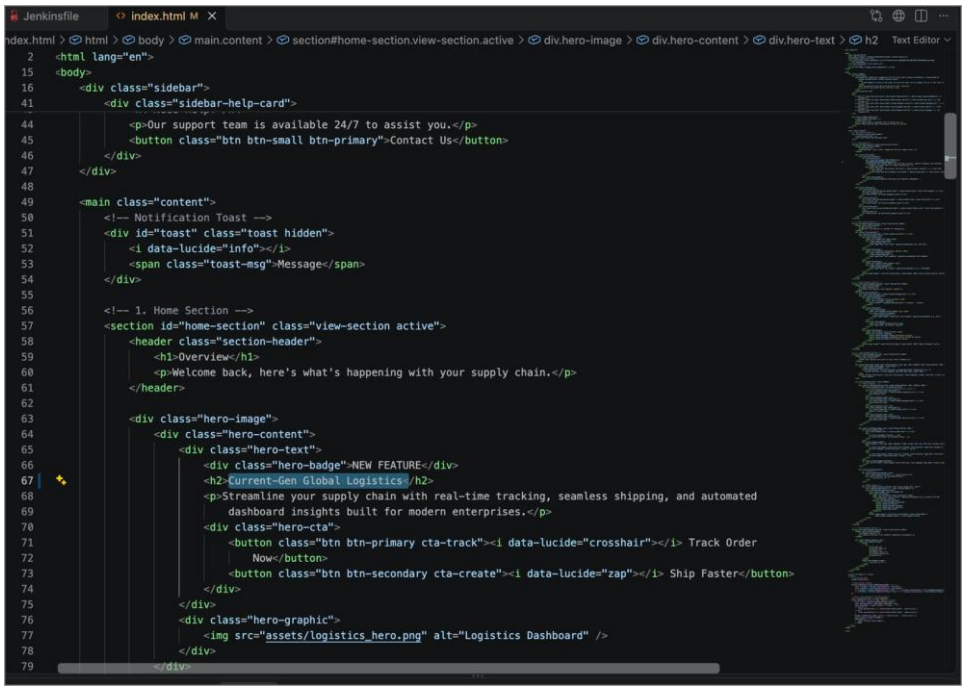


Figure 5.30: Heading edited in public/index.html (file shown as modified)

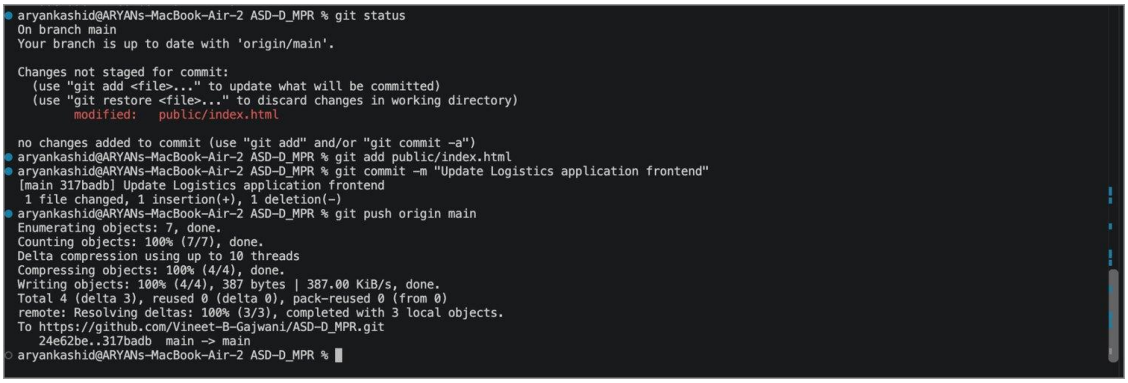


Figure 5.31: git status, git add, git commit and git push for the change (commit 317badb)

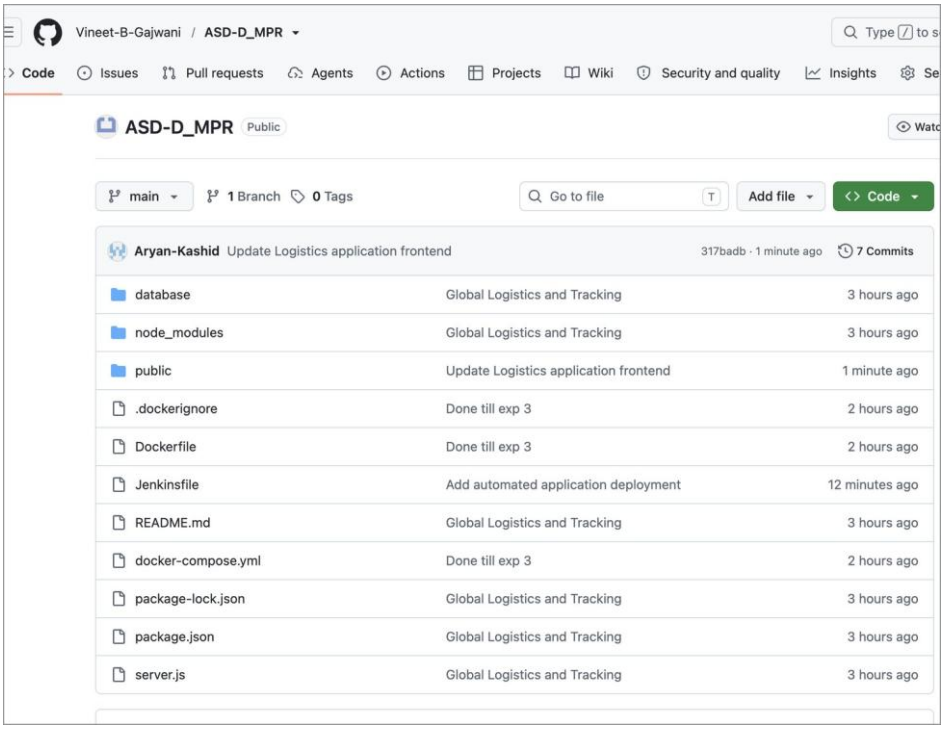


Figure 5.32: GitHub repository after the push: Update Logistics application frontend

Jenkins build 8 (4 October 2026, 19:55) then checked out revision 317badb, the commit just pushed (Figures 5.33 and 5.34), rebuilt the image and redeployed the stack, and the application at `http://localhost:3000` showed the new heading (Figure 5.35). The build was started from the Jenkins interface by a team member after the push, so the trigger was manual in this implementation; a webhook or repository polling that starts the build automatically is listed under future scope, since a Jenkins server running on a developer laptop is not reachable from GitHub without a public address or tunnel.

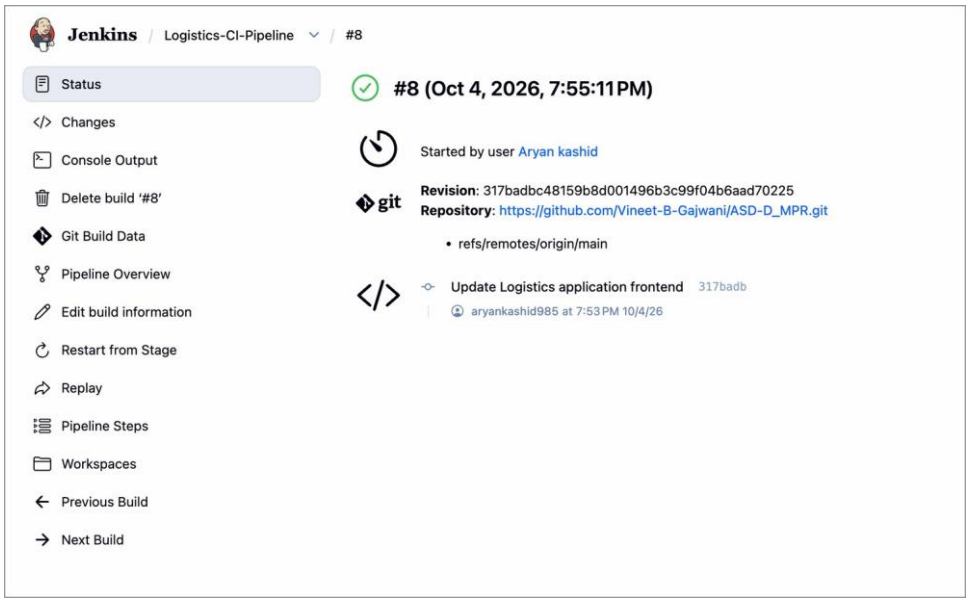


Figure 5.33: Build 8 started for commit 317badb: Update Logistics application frontend

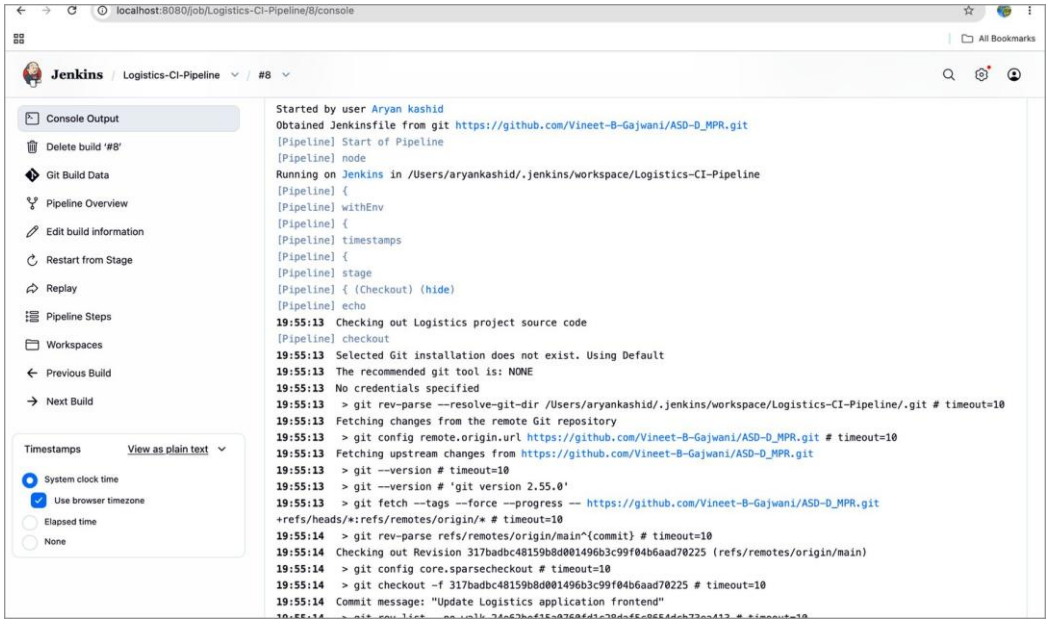


Figure 5.34: Build 8 console: checkout of revision 317badb

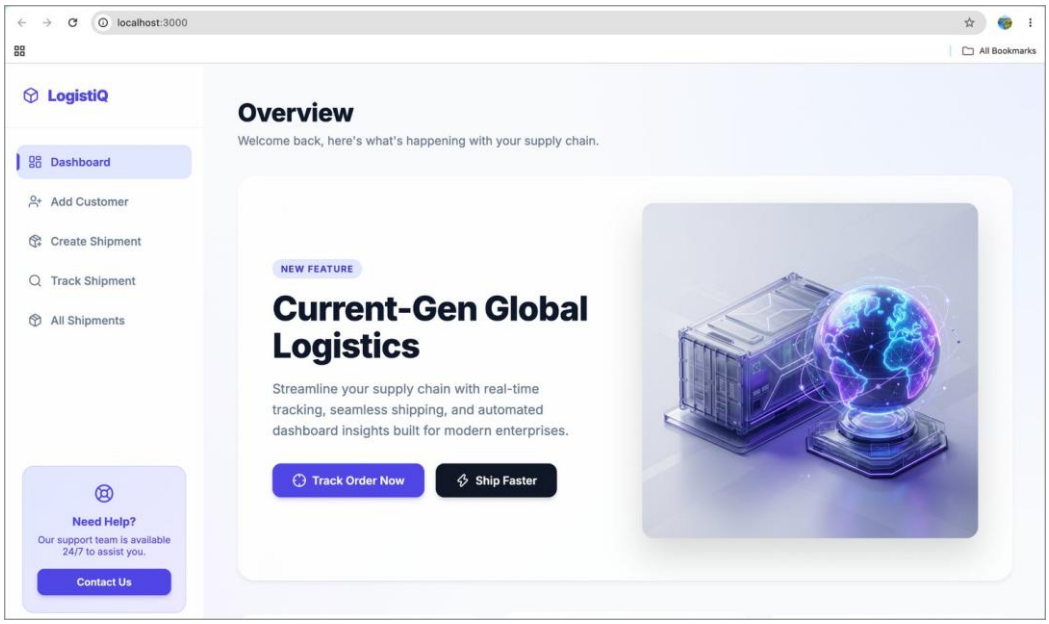


Figure 5.35: Application after deployment: heading now Current-Gen Global Logistics

5.6 Monitoring (Experiment 7)

Monitoring was implemented with Nagios Core 4.5.12, which uses a state-based model: it asks, at a regular interval, whether each service is working now. Nagios was added to the Compose file as a third service, `logistics-nagios`, from the image *jasonrivers/nagios*, with its web interface published on port 8081 (Jenkins already uses 8080). Figure 5.36 shows the stack with the Nagios container started, and Figure 5.37 shows the Nagios web interface.

```
Vineet Gajwani@LAPTOP-HMASL84U MINGW64 ~/OneDrive/Desktop/ASD&D_MPR (main)
$ docker compose ps
NAME              IMAGE                          COMMAND                  SERVICE   CREATED          STATUS          PORTS
logistics-app     asdd_mpr-app                   "docker-entrypoint.s…"   app       25 hours ago     Up 3 minutes    0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp
logistics-mysql   mysql:8.0                      "docker-entrypoint.s…"   mysql     25 hours ago     Up 3 minutes    3306/tcp, 33060/tcp
logistics-nagios  jasonrivers/nagios:latest      "/usr/local/bin/star…"   nagios    8 seconds ago    Up 8 seconds    0.0.0.0:8081->80/tcp, [::]:8081->80/tcp
```

Figure 5.36: docker compose ps: application, MySQL and Nagios containers

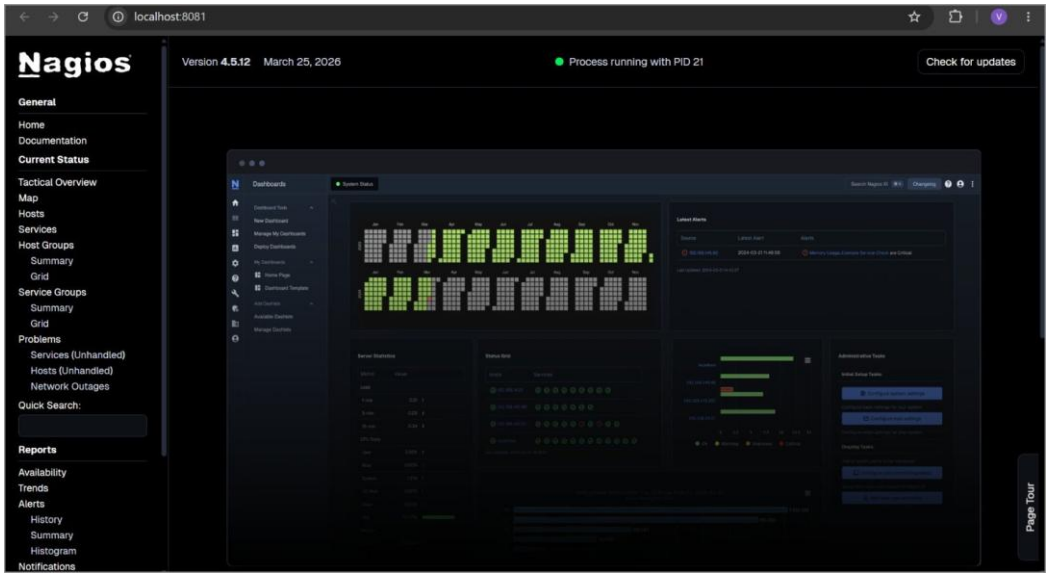


Figure 5.37: Nagios Core web interface at localhost:8081

Two object definitions were then added to describe the project: a host for the application container with an HTTP service check, and a host for the database container with a TCP check on port 3306. Before reloading Nagios, the configuration was validated with the pre-flight command `nagios -v` run inside the container. The first check reported 7 services and 1 host with 0 warnings and 0 errors (Figure 5.38). After the project hosts were added it reported 9 services and 3 hosts and 0 errors; its 6 warnings only say that the new services have no contact group or notification period, which is not an error because no notifications were configured (Figure 5.39).

```
PS C:\Users\Vineet Gajwani\OneDrive\Desktop\ASD&D_MPR> docker exec logistics-nagios /opt/nagios/bin/nagios -v /opt/nagios/etc/nagios.cfg
Last Modified: 2026-03-25
License: GPL

Website: https://www.nagios.org
Reading configuration data...
   Read main config file okay...
   Read object config files okay...

Running pre-flight check on configuration data...

Checking objects...
        Checked 7 services.
        Checked 1 hosts.
        Checked 1 host groups.
        Checked 0 service groups.
        Checked 1 contacts.
        Checked 1 contact groups.
        Checked 23 commands.
        Checked 5 time periods.
        Checked 0 host escalations.
        Checked 0 service escalations.
Checking for circular paths...
        Checked 1 hosts
        Checked 0 service dependencies
        Checked 0 host dependencies
        Checked 5 timeperiods
Checking global event handlers...
Checking obsessive compulsive processor commands...
Checking misc settings...

Total Warnings: 0
Total Errors:   0

Things look okay - No serious problems were detected during the pre-flight check
```

Figure 5.38: Nagios pre-flight check of the bundled configuration: 0 warnings, 0 errors

```
PS C:\Users\Vineet Gajwani\OneDrive\Desktop\ASD&D_MPR> docker exec logistics-nagios /opt/nagios/bin/nagios -v /opt/nagios/etc/nagios.cfg

Nagios Core 4.5.12
Copyright (c) 2009-present Nagios Core Development Team and Community Contributors
Copyright (c) 1999-2009 Ethan Galstad
Last Modified: 2026-03-25
License: GPL

Website: https://www.nagios.org
Reading configuration data...
   Read main config file okay...
   Read object config files okay...

Running pre-flight check on configuration data...

Checking objects...
Warning: Service 'Logistics Web Application' on host 'logistics-app' has no default contacts or contactgroups defined!
Warning: Service 'Logistics Web Application' on host 'logistics-app' has no notification time period defined!
Warning: Service 'MySQL Database Port' on host 'logistics-mysql' has no default contacts or contactgroups defined!
Warning: Service 'MySQL Database Port' on host 'logistics-mysql' has no notification time period defined!
        Checked 9 services.
Warning: Host 'logistics-app' has no default contacts or contactgroups defined!
Warning: Host 'logistics-mysql' has no default contacts or contactgroups defined!
        Checked 3 hosts.
        Checked 1 host groups.
        Checked 0 service groups.
        Checked 1 contacts.
        Checked 1 contact groups.
        Checked 23 commands.
        Checked 5 time periods.
        Checked 0 host escalations.
        Checked 0 service escalations.
Checking for circular paths...
        Checked 3 hosts
        Checked 0 service dependencies
        Checked 0 host dependencies
        Checked 5 timeperiods
Checking global event handlers...
Checking obsessive compulsive processor commands...
Checking misc settings...

Total Warnings: 6
Total Errors:   0

Things look okay - No serious problems were detected during the pre-flight check
```

Figure 5.39: Nagios pre-flight check with the project hosts: 9 services, 3 hosts, 0 errors

The service status page (Figure 5.40) shows three hosts up and nine services, of which eight are OK. The *Logistics Web Application* check returns HTTP OK with 17,135 bytes in 0.029 seconds, and the *MySQL Database Port* check returns TCP OK on port 3306. The single WARNING belongs to the *HTTP* check of the bundled *localhost* host, which monitors the Nagios web server itself: it receives status 401 Unauthorized because that interface is password protected. It is a reading about the bundled configuration, not a fault in the logistics application.

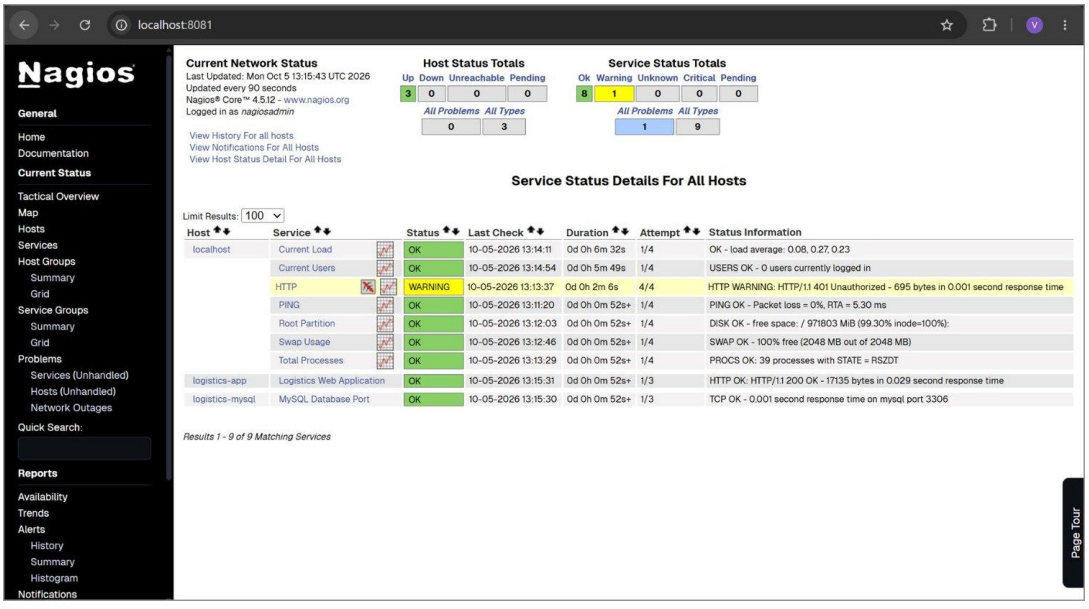


Figure 5.40: Nagios service status for all hosts

6. Testing and Verification

The project did not include an automated unit-test suite. Testing was therefore carried out at the level of the delivery pipeline and the running system, using checks that were run both by hand and by Jenkins. Each check below is supported by a screenshot in the report.

6.1 Verification Checks

ID	Check	Method	Expected result	Result
V1	Dependencies install from the lock file	`npm ci` on the Jenkins host (Figure 5.16)	83 packages added without error	Pass
V2	Server code is syntactically valid	`node --check server.js` (Figure 5.16; stage Validate Application)	No syntax error	Pass
V3	Compose file is valid	`docker compose config --quiet` (Figure 5.16; stage Validate Docker Compose)	No output, exit code 0	Pass
V4	Image builds	`docker build` and `docker compose build` (Figures 5.7, 5.12, 5.24)	Image created	Pass
V5	Container starts and connects to MySQL	`docker logs` (Figure 5.11)	Server running; Connected to MySQL Database!	Pass
V6	Database schema is loaded	`SHOW TABLES` (Figure 3.2)	Ten application tables listed	Pass
V7	Application is reachable	Browser at `localhost:3000` (Figure 4.2)	Dashboard displayed	Pass
V8	Pipeline builds the committed code	Jenkins builds 5 to 8 (Figures 5.22, 5.26, 5.34)	Status SUCCESS	Pass
V9	Deployment starts both containers	`docker compose ps` in build 7 (Figure 5.28)	App and MySQL Up	Pass
V10	A code change reaches the running app	Heading change, build 8 (Figures 5.30 to 5.35)	New heading visible after the build	Pass
V11	Monitoring configuration is valid	`nagios -v` (Figures 5.38, 5.39)	0 errors	Pass
V12	Web application and database are up	Nagios service status (Figure 5.40)	HTTP OK and TCP OK	Pass

Table 6.1: Verification checks and results

6.2 Pipeline Executions

Build	Time (4 Oct 2026)	Result	Notes
#1	17:53	Failed	Pipeline and environment being configured
#2	18:00	Failed	Pipeline and environment being configured
#3	18:03	Failed	Pipeline and environment being configured
#4	18:07	Failed	Pipeline and environment being configured
#5	18:13	Success	First successful CI build: checkout, validation and image build
#6	18:16	Success	Repeat run of the CI pipeline

Build	Time (4 Oct 2026)	Result	Notes
#7	19:41	Success	First build with Deploy and Verify stages; duration 1 min 33 s
#8	19:55	Success	Deployed the dashboard heading change (commit 317badb)

Table 6.2: Jenkins build history

6.3 Observations from Security and Dependency Checks

The `npm ci` run on the Jenkins host reported 4 vulnerabilities in dependencies (1 low, 1 moderate and 2 high), which `npm audit` can review. These were not changed during the project because no application code was modified, but they are recorded here as an item for follow-up in the future scope.

7. Results and Discussion

7.1 Outcomes against Objectives

Objective	Outcome
Customer, shipment, tracking and overview functions	Delivered by the existing application, which runs unchanged in Docker
Plan in Jira with epic, stories, sprint and board	1 epic, 6 stories, 1 sprint; stories moved across To Do, In Progress and Done
Source under Git and on GitHub	Repository ASD-D_MPR on GitHub with the history of each increment
Docker image and one-command stack	Image logistics-app built; `docker compose up --build` starts the application and MySQL
Declarative Jenkins pipeline from the repository	Eight stages; builds 5 to 8 successful
Automated deployment	Deploy and Verify stages; a code change deployed by build 8
Monitoring with Nagios	Nagios container in the stack; HTTP and TCP checks for the application and database all OK

Table 7.1: Results against objectives

7.2 The Feedback Loop in Practice

The practices described in Chapter 2 were visible in three episodes. First, the first four Jenkins executions failed while the pipeline and the agent environment were being configured. Each failure was read in the console log, the configuration was adjusted and the job was run again, and build 5 was the first success: this is the inspect-and-adapt cycle in its simplest form, and it was possible only because the pipeline gave feedback within minutes.

Second, when Compose first started the stack, the application container tried to connect to MySQL before the database had finished initialising and logged *ECONNREFUSED*; it then connected once MySQL accepted connections (Figure 5.14). The episode showed that the application depends on the start-up order of its services, and the point was recorded as an improvement for the Compose configuration (Section 8.2).

Third, Nagios reported a WARNING for the bundled *HTTP* check of its own localhost host. Investigation of the status text (401 Unauthorized) showed that the check was reaching a password-protected page and that the logistics application itself was healthy, so the reading was documented rather than treated as a defect.

7.3 Challenges and Resolutions

Challenge	Resolution
Application container could not reach MySQL when started alone with `localhost`	Both services run in Docker Compose and the application connects to the host name `mysql` on the shared network
Application started before the database was ready	The connection was retried and succeeded once MySQL was initialised; start-up ordering was recorded as an improvement
First four Jenkins executions failed	The pipeline and agent environment were corrected, including setting `PATH` in the Jenkinsfile so that Docker, Node.js and Git are found, and build 5 succeeded

Challenge	Resolution
Port 8080 already used by Jenkins	Nagios web interface published on host port 8081
Nagios warned that new services had no contact group	Recognised as a notification setting, not a fault; configuration passed with 0 errors
HTTP WARNING on the bundled Nagios localhost host	Identified as a 401 from the password-protected Nagios page, unrelated to the application
Large first Docker build (176 s)	Base image pulled once and then cached; later builds reused the cache

Table 7.2: Challenges met and how they were resolved

7.4 Limitations

- **Manual build trigger.** Jenkins builds were started from the Jenkins interface. A GitHub webhook or SCM polling would make every push start a build automatically, but needs a Jenkins address that GitHub can reach.
- **Repository hygiene.** The initial commit (993 files) included the `node_modules` folder and the `.env` file, which holds database credentials. These should be removed from tracking with `git rm --cached`, listed in a `.gitignore` file, and the database password should be changed.
- **No automated unit tests.** Verification relies on syntax, configuration and deployment checks and on manual use of the interface; the pipeline has no test stage.
- **Monitoring depth.** Nagios checks availability only. It does not record request rates, latency or resource use over time.
- **Single host.** Jenkins, the application and Nagios run on the same machine, so a host failure would remove both the system and the means of observing it.
- **Jira status.** In the final board capture the monitoring story was still in To Do, although the monitoring experiment was completed; the board should be updated to match.

8. Conclusion and Future Scope

8.1 Conclusion

The Global Logistics Tracking System was taken from a local Node.js, Express and MySQL project to a version-controlled, containerised, automatically built and deployed and monitored system, without rebuilding the application. The work was organised as an Agile project: one epic and six stories in a Jira sprint, one working increment per story, a visible board and a Definition of Done that required evidence for each story.

Git and GitHub provided the shared history. Docker and Docker Compose made the environment a versioned artefact, so that the application and its database start with one command. The Jenkins pipeline turned validation, build and deployment into repeatable stages, and build 8 showed a change in the dashboard heading passing from the editor to the running container. Nagios added a continuous check of the application and the database. The practical lesson of the project is that each practice makes the next one possible: the repository is what the pipeline builds, the image is what Compose runs, the running stack is what monitoring observes and the backlog is where what monitoring finds returns.

8.2 Future Scope

- Add a GitHub webhook or SCM polling so that every push starts a Jenkins build automatically, and use a multibranch pipeline with one branch per Jira issue key.
- Clean the repository: add `.gitignore` and `.env.example`, remove `node_modules` and `.env` from tracking, rotate the database password and store credentials in Jenkins credentials instead of files.
- Add Compose health checks and `depends_on` with `condition: service_healthy` so that the application starts only after MySQL accepts connections.
- Add an `/api/health` endpoint that reports database connectivity, and use it as a Jenkins smoke test and as a Nagios check.
- Add an automated test stage (for example Jest with Supertest) and publish the results in Jenkins.
- Add Prometheus with the `prom-client` library and Grafana dashboards for request rate, response time, memory and CPU, complementing the availability checks of Nagios.
- Use a smaller base image such as `node:20-alpine` and a non-root user to reduce the image size (currently 1.6 GB) and the attack surface.
- Add user authentication, role-based access and customer-facing tracking links.

References

1. Docker Inc., *Docker Documentation*, https://docs.docker.com/
2. Docker Inc., *Docker Compose Documentation*, https://docs.docker.com/compose/
3. Jenkins Project, *Jenkins User Documentation: Pipeline*, https://www.jenkins.io/doc/book/pipeline/
4. Git Project, *Git Documentation*, https://git-scm.com/doc
5. GitHub, *GitHub Docs*, https://docs.github.com/
6. Atlassian, *Scrum with Jira*, https://www.atlassian.com/agile/scrum
7. Nagios Enterprises, *Nagios Core Documentation*, https://www.nagios.org/documentation/
8. OpenJS Foundation, *Node.js Documentation*, https://nodejs.org/docs/
9. Express.js, *Express Documentation*, https://expressjs.com/
10. Oracle, *MySQL 8.0 Reference Manual*, https://dev.mysql.com/doc/refman/8.0/en/
11. K. Schwaber and J. Sutherland, *The Scrum Guide*, https://scrumguides.org/

Appendix A: Running the Project

A.1 Prerequisites

- Docker with Docker Compose.
- Git.
- Optional: Node.js 20 and MySQL 8.0 to run the application without Docker.
- Optional: Jenkins 2.568 or later for the pipeline.

A.2 Run with Docker Compose

```
git clone https://github.com/Vineet-B-Gajwani/ASD-D_MPR.git
cd ASD-D_MPR
# create .env with DB_HOST, DB_USER, DB_PASSWORD, DB_NAME and PORT
docker compose up --build
```

Open `http://localhost:3000` for the application and `http://localhost:8081` for the Nagios web interface. In Compose the database host must be `mysql`.

A.3 Useful commands

```
docker compose ps                 # list running containers
docker logs logistics-app         # application log
docker exec -it logistics-mysql mysql -uroot -p
docker compose down               # stop and remove the stack
```

A.4 Run the pipeline

1. Start Jenkins and open `http://localhost:8080`.
2. Create a Pipeline job and choose *Pipeline script from SCM*, with Git, the repository URL, branch `*/main` and script path `Jenkinsfile`.
3. Select *Build Now* and read the stages and the console output.
4. After the build, open `http://localhost:3000` to see the deployed application.